



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА



Кафедра цифровой трансформации

Институт информационных технологий

Презентации по лекционным материалам по дисциплине «Разработка баз данных»

Направления подготовки:

Уровень: бакалавриат

Форма обучения: очная

01.03.04 Прикладная математика
09.03.03 Прикладная информатика
09.03.04 Программная инженерия
09.03.01 Информатика и вычислительная техника

Лекция №7. Транзакции. Оптимизация SQL запросов.

Лектор
Исаева Мария Владимировна
Кандидат технических наук,
доцент кафедры
цифровой трансформации

2025/2026 учебный год

СВЕДЕНИЯ О ДИСЦИПЛИНЕ

Лекции ведут:

- Исаева Мария Владимировна, к.т.н., доцент каф. Цифровой трансформации
- Дзгоев Алан Эдуардович, к.т.н., доцент каф. Цифровой трансформации
(Dzgoviev@mirea.ru / *При обращении в теме письма указывайте номер группы*)
- Миронов Антон Николаевич, Старший преподаватель каф. Цифровой трансформации
- Семькина Наталья Александровна, д.т.н., профессор каф. Цифровой трансформации
- Резеньков Роман Николаевич, к.т.н., доцент каф. Цифровой трансформации

Объём **аудиторной** работы по дисциплине в 5 семестре:

- Лекции – 16 часов;
- Практические занятия – 48 часа;
- Аттестация – экзамен.

Допуск к экзамену:

- посещение лекций и практических работ;
- выполнение в установленный срок всех практических работ.

подробности в памятке по дисциплине (следующие слайды)

Выполнение практических заданий

1. Все материалы курса на <https://online-edu.mirea.ru> в разделе **Разработка баз данных: доступ к материалам курса открывается по расписанию занятий.**

2. Доступ с аккаунта МИРЭА.

3. Все создаваемые файлы прикладываются к заданию.

4. Учёт выполненных работ.

5. Общение с преподавателем через отзывы в заданиях.

Курс	Л4_ % кликов по кнопке "контроль присутствия"	Л5_ % кликов по кнопке "контроль присутствия"	Л6_04-05_ % присутствия	Л7_18-05_ % присутствия	Л8_01-06_ % присутствия	%СРЗНАЧ участия в лекциях	Задание:Практика 1_Законодательная и нормативно-техническая база стандартизации в РФ	Задание:Практика 2_Нормативные документы по стандартизации ИТ	Задание:Практика 3_ч1_Классификаторы в области ИТ	Задание:Практика 3_ч2_Классификаторы в области ИТ	Задание:Практика 3_ч3_Классификаторы в области ИТ	Задание:Практика 4_ч1_Создание технического задания в соответствии с ГОСТ 34.602-2020
0	0	50				6,3	-	-	-	-	-	-
0	0	50		100	100	31,3	-	1	1	1	1	1
0	0	0				0	-	-	-	-	-	-
00	0	50	100		100	56,3	1	1	1	1	1	1
100	100	100	100	100	100	87,5	1	1	1	1	1	1
0	0	0				0	-	-	-	-	-	-
0	50	0				6,3	1	-	-	-	-	-
100	0	100		100		56,3	1	1	1	1	1	1
0	0	0				12,5	1	1	-	-	-	-
0	0	0				0	-	-	-	-	-	-
0	0	0				0	-	-	-	-	-	-
100	0	0				18,8	-	-	-	-	-	-
0	100	100	100	100	100	68,8	1	1	1	1	1	1
0	0	50				18,8	-	-	-	-	-	-
0	0	0	100	100	100	50	1	1	1	1	1	1
0	0	0				0	-	-	-	-	-	-
0	100	100	100	100	100	100	1	1	1	1	1	1
0	0	0				0	-	-	-	-	-	-
100	0	0				18,8	1	1	1	1	-	-
50	0	50				18,8	-	-	-	-	-	-
0	0	0				0	-	-	-	-	-	-
0	0	0				0	-	-	-	-	-	-
0	0	0			100	12,5	1	1	1	1	1	1
100	100	0				25	1	1	-	-	-	-
0	0	0				0	-	-	-	-	-	-
100	100	100	100	100	100	100	1	1	1	1	1	1



Рекомендуемая литература

1. Новиков Б.А. Основы технологий баз данных: учебное пособие / Б.А. Новиков, Е.А. Горшкова, Н.Г. Графеева; под.ред. Е.В. Рогова. – 2-е изд. – М.: ДМК Пресс, 2020. – 582 с. Режим доступа: <https://postgrespro.ru/education/books/dbtech>
2. Моргунов Е.П. PostgreSQL. Основы языка SQL: учеб. Пособие / Е.П. Моргунов; под. ред. Е.В. Рогова, П.В. Лузанова. – СПб.: БХВ-Петербург, 2018. – 336 с.: ил. Режим доступа: <https://postgrespro.ru/education/books/sqlprimer>
3. Комаров В.И. Путеводитель по базам данных. – М.: ДМК-Пресс, 2024. – 520 с. Режим доступа: <https://edu.postgrespro.ru/dbguide.pdf>
4. Смирнов М.В. Проектирование баз данных [Электронный ресурс]: Конспект лекций / Смирнов М.В. - М., МИРЭА – Российский технологический университет, 2020 – 1 электрон. Опт. диск (CD-ROM). Режим доступа: <https://e.lanbook.com/book/163892/>
5. Чистякова М.А. Проектирование и эксплуатация баз данных [Электронный ресурс]: Учебно-методическое пособие / Чистякова М.А., Иванова И.А., Котилевец И.Д. – М.: МИРЭА – Российский технологический университет, 2021. – 1 электрон. Опт. диск (CD-ROM). Режим доступа: <https://e.lanbook.com/book/176572/>
6. Братусь Н.В. Базы данных. Часть 1 [Электронный ресурс]: Практикум / Братусь Н.В., Маличенко С.В., Матчин В.Т. – М.: МИРЭА – Российский технологический университет, 2024. – 1 электрон. опт. диск (CD-ROM) – Режим доступа: <https://ibc.mirea.ru/books/SHARE/5859/>
7. Моргунов Е. П. PostgreSQL. Профессиональный SQL : учеб. пособие / Е. П. Моргунов; под ред. Е. В. Рогова. – М.: ДМК Пресс, 2025. – 444 с. Режим доступа: <https://postgrespro.ru/education/books/advancedsql>
8. Рогов Е. В. PostgreSQL 17 изнутри. – М.: ДМК Пресс, 2025. – 668 с. Режим доступа: <https://postgrespro.ru/education/books/internals>
9. Введение в системы баз данных. : Пер. с англ. / К. Дж. Дейт .— М. : Изд. дом "Вильямс", 2001 .— 1072 с. https://ibc.mirea.ru/books/search/?search_field=Дейт&page=3

Неделя	Лекции	Практические работы	Дедлайны, текущие и контрольные мероприятия
1	Лекция 1. Тема: Реляционная модель данных и базовые операции SQL. (2 часа)	Занятие 1. ПРАКТИЧЕСКАЯ РАБОТА №1. Создание БД и запросы на выборку данных (2 часа)	
2		Занятие 2. ПРАКТИЧЕСКАЯ РАБОТА №1. (2 часа)	Дедлайн по защите отчета по практике №1.
		Занятие 3. ПРАКТИЧЕСКАЯ РАБОТА №2. Многотабличные запросы, использование операций объединения, пересечения, разности (2 часа)	
3	Лекция 2. Тема: Многотабличные запросы и теоретико-множественные операции. (2 часа)	Занятие 4. ПРАКТИЧЕСКАЯ РАБОТА №2. (2 часа)	
4		Занятие 5. ПРАКТИЧЕСКАЯ РАБОТА №2. (2 часа)	Дедлайн по защите отчета по практике №2.
		Занятие 6 ПРАКТИЧЕСКАЯ РАБОТА №3 Использование подзапросов и CTE (2 часа)	
5	Лекция 3. Использование подзапросов и агрегатных выражений (2 часа)	Занятие 7. ПРАКТИЧЕСКАЯ РАБОТА №3 (2 часа)	Тест №1. (по лекциям №1-2).
6		Занятие 8. ПРАКТИЧЕСКАЯ РАБОТА №3 (2 часа).	Дедлайн по защите отчета по практике №3.
		Занятие 9. ПРАКТИЧЕСКАЯ РАБОТА №4 Использование оконных функций и функций ранжирования (2 часа)	
7	Лекция 4. Тема: SQL. Оконные функции и аналитические запросы (2 часа)	Занятие 10. ПРАКТИЧЕСКАЯ РАБОТА №4 (2 часа)	Дедлайн по защите отчета по практике №4.
8		Занятие 11. ПРАКТИЧЕСКАЯ РАБОТА №4 (2 часа)	Дедлайн по защите отчета по практике №4.
		Занятие 12. ПРАКТИЧЕСКАЯ РАБОТА №5 (2 часа)	
9	Лекция 5. Операторы модификации данных, объекты базы данных (2 часа)	Занятие 13. ПРАКТИЧЕСКАЯ РАБОТА №5 (2 часа)	Дедлайн по защите отчета по практике №5.

План – график лекций и практических занятий

10		Занятие 14. ПРАКТИЧЕСКАЯ РАБОТА №5 (2 часа)	Дедлайн по защите отчета по практике №5.
		Занятие 15. ПРАКТИЧЕСКАЯ РАБОТА №6 (2 часа)	
11	Лекция 6. Управление данными: триггеры, курсоры (2 часа)	Занятие 16. 2 часа	Контрольная работа №1.
12		Занятие 17. ПРАКТИЧЕСКАЯ РАБОТА №6 (2 часа)	Дедлайн по защите отчета по практике №6.
		Занятие 18. ПРАКТИЧЕСКАЯ РАБОТА №7 Оптимизация запросов (2 часа)	Тест №2. (по лекциям №3-5).
13	Лекция 7. Оптимизация запросов (2 часа)	Занятие 19. ПРАКТИЧЕСКАЯ РАБОТА №7 (2 часа)	
14		Занятие 20. ПРАКТИЧЕСКАЯ РАБОТА №7 (2 часа)	Дедлайн по защите отчета по практике №7.
		Занятие 21. ПРАКТИЧЕСКАЯ РАБОТА №8 Хранение неструктурированных данных (2 часа)	
15	Лекция 8. Администрирование баз данных и хранение неструктурированных данных (2 часа) Тест №3. (по лекциям №6-7).	Занятие 22. ПРАКТИЧЕСКАЯ РАБОТА №8 (2 часа)	
16		Занятие 23. ПРАКТИЧЕСКАЯ РАБОТА №8 (2 часа)	Дедлайн Практики №8.
		Занятие 24. 2 часа	Контрольная работа №2.
Итого	16 часов	48 часов	

ПАМЯТКА

о правилах обучения дисциплины

«Разработка баз данных» (2025-2026, I семестр)

№	Наименование	Формат	Период проведения	Баллы (макс.)
1	Защита практических работ (8 практик)	Очно	Сентябрь 2025 – Декабрь 2025	48 (6 баллов за 1 практику)
2	Контрольный тест №1 (по лекциям №1 и №2)	Очно, СДО	Октябрь 2025	5
3	Контрольный тест №2 (по лекциям №3 - №5)	Очно, СДО	Ноябрь 2025	5
4	Контрольная работа №1	Очно, СДО	Ноябрь 2025	10
5	Контрольный тест №3 (по лекциям №6 - №8)	Очно, СДО	Декабрь 2025	6
6	Контрольная работа №2	Очно, СДО	Декабрь 2025	10
7	Посещение (лекции 8 занятий, практики 24 занятия)	Очно	Сентябрь– Декабрь 2025	16 б. 0,5 баллов за посещения 1 лекции и практики (0,5·32 пар)
Максимальная сумма баллов за активность по дисциплине в течение учебного семестра				100

ДИСЦИПЛИНА	Разработка баз данных (укажите полное наименование дисциплины без сокращений)
ИНСТИТУТ	информационных технологий (укажите название учебного института)
КАФЕДРА	Цифровой трансформации (укажите полное наименование кафедры)
Уровень обучения	Бакалавриат
Аудиторные часы	16/0/48 (укажите количество часов лекционных, лабораторных и практических)

Лекции 16 часов.

Обязательное посещение всех лекций.

В тестах в рамках мероприятий текущего контроля вопросы из лекций

Практические работы (8 работ)

Обязательное посещение всех практических занятий.

Допуск к экзамену – очная защита всех практических работ в течение семестра. Защита отчёта до дедлайна

Защита практической работы после дедлайна – 0 баллов, но работа засчитывается.

Если есть справка по перенесённой болезни, заверенная в учебном отделе, то эту справку необходимо принести и показать преподавателю по практике. В таком случае назначается пересдача пропущенной практической работы в день консультаций (важно: студент защищает именно ту практическую работу, которую он пропустил по болезни, согласно датам в справке). Если студент пропустил практические работы без уважительной причины и, соответственно, работу не защитил, то на экзамен он не допускается.

Если студент загрузил отчет до дедлайна, но не защитил, то такой отчет не засчитывается студенту, т.е. не сдал.

Дополнительного времени на защиту практических работ не выделяется.

Выполненные работы студент загружает в СДО в формате pdf. Фон области построения модели – белый (в отчёте).

№	Наименование	Формат	Период проведения	Баллы (макс.)
1	Защита практических работ (8 практик)	Очно	Сентябрь 2025 – Декабрь 2025	48 (6 баллов за 1 практику)
2	Контрольный тест №1 (по лекциям №1 и №2)	Очно, СДО	Октябрь 2025	5
3	Контрольный тест №2 (по лекциям №3 - №5)	Очно, СДО	Ноябрь 2025	5
4	Контрольная работа №1	Очно, СДО	Ноябрь 2025	10
5	Контрольный тест №3 (по лекциям №6 - №8)	Очно, СДО	Декабрь 2025	6
6	Контрольная работа №2	Очно, СДО	Декабрь 2025	10
7	Посещение (лекции 8 занятий, практики 24 занятия)	Очно	Сентябрь– Декабрь 2025	16 б. 0,5 баллов за посещения 1 лекции и практики (0,5·32 пар)
Максимальная сумма баллов за активность по дисциплине в течение учебного семестра				100

Если студент не загрузил отчет до дедлайна, то загрузка после дедлайна станет недоступна.

Контрольные тесты

Тесты студенты выполняют в СДО на паре.

Баллы минусуются, если по базовым вопросам студент отвечает не правильно.

Проходного балла нет, сколько студент набрал столько и остается.

Контрольный тест №1. (Лекции 1, 2) проводится на 5-м практическом занятии в начале пары (20 минут).

Контрольный тест №2. (Лекции 3, 4, 5) проводится на 18-м практическом занятии в начале пары (20 минут)..

Контрольный тест №3 (Лекции №6-7) проводится на 8-й лекции в начале пары (20 минут).

Контрольные работы

Выполняются на 16 и 24 практических занятиях в аудитории.

Студент получает задание на паре.

Время выполнения контрольной работы – 1 час.

Экзамен

Допуск к экзамену – защита всех практических работ в течение семестра очно. В случае, если студент не сдал какие-либо практики, у него есть возможность сдать их во время экзамена, если успеет. В противном случае, отправляется на пересдачу (необходимость сдачи работ сохраняется).

Если студент набрал меньше 50 % от общей суммы баллов, то экзамен будет проходить по билету (2теоретических вопроса + 1 задача).

Если студент набрал больше 50% от суммы максимальной суммы баллов, то сдаёт тестирование на экзамене. + добавляются баллы за активность.

За тест студент может набрать 20 баллов, за которые можно получить:

- От 10 до 14 баллов — оценка «3»;
- От 15 до 19 баллов — оценка «4»;
- 20 баллов — оценка «5».

К баллам за тест прибавляются доп. баллы, которые студент получает в течение семестра по данным критериям:

- Набрано 55-64 балла за семестр — 1 доп.;
- Набрано 65-74 балла за семестр — 2 доп.;
- Набрано 75-84 балла за семестр — 3 доп.;
- Набрано 85-94 балла за семестр — 4 доп.;
- Набрано 95-100 балла за семестр — 5 доп.

В случае, если студент сдал тест на «2», то отправляется на пересдачу без учета доп. баллов.


```

graph TD
    Данные[Данные] --> Структурированные[Структурированные БД (4 семестр)]
    Данные --> Полуструктурированные[Полуструктурированные БД (5 семестр)]
    Данные --> Неструктурированные[Неструктурированные БД (магистратура, 2 семестр)]

    Структурированные --> СУБД[СУБД]
    Структурированные --> SQL[SQL]
    Структурированные --> Архитектура[Архитектура СУБД]

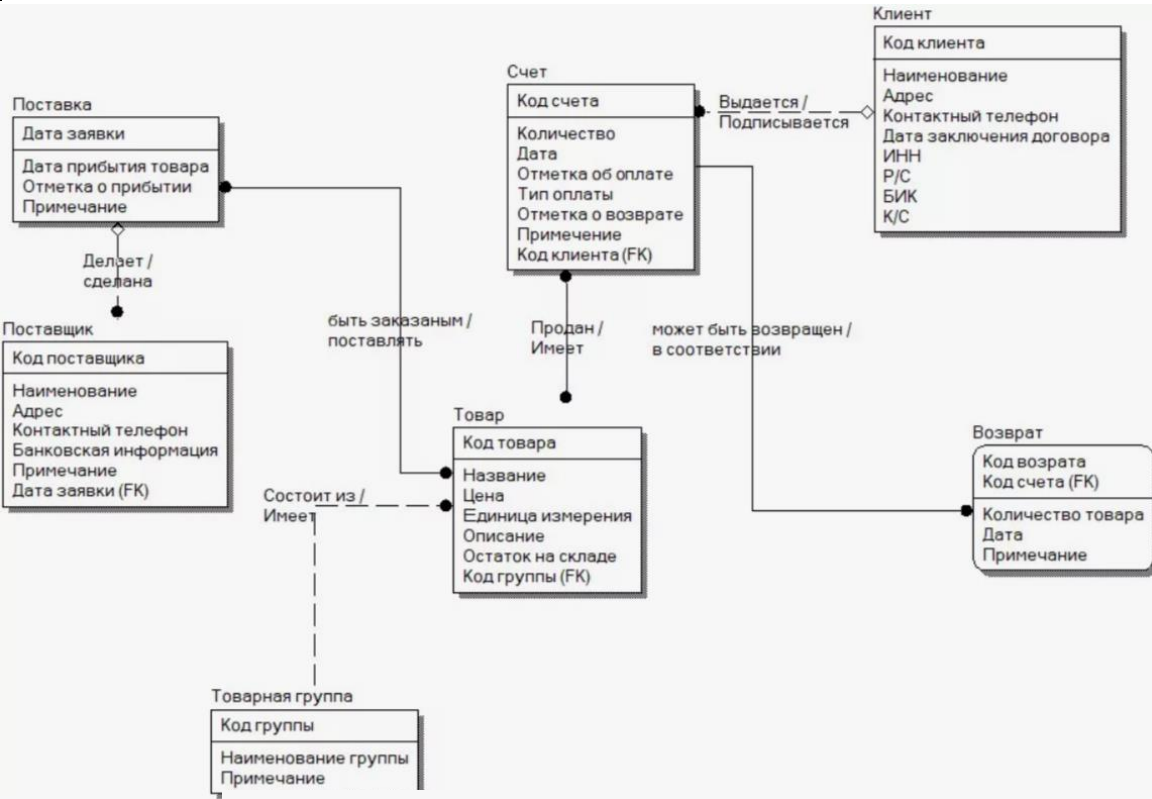
    SQL --> Классические["Классические  
Нормализация"]
    SQL --> Современные["Современные  
Оптимизация (немного)  
шардирование и кэширование"]
    SQL --> Теорема[Теорема Брюера, репликация, распределенные БД]

    Полуструктурированные --> NoSQL[NoSQL]
    NoSQL --> Документориентированные[Документориентированные (Maria DB)]
    NoSQL --> TSBD[TSBD, Redis, Clickhouse, MONQ, Tinkoff SAGE]
    NoSQL --> ИТД[и т.д.]

    Неструктурированные --> Хранилища[Хранилища]
    Хранилища --> Файловые[Файловые системы (HDFS, ZFS)]
    Хранилища --> S3[S3, Объектное]
    Хранилища --> State[State Full, State Less]
    Хранилища --> MapReduce[MapReduce]
    Хранилища --> REST[REST API, Сервисные шины]
    Хранилища --> Управление[Управление данными в системе]
    Хранилища --> МестоСУБД[Место СУБД в информационной архитектуре предприятия]
    Хранилища --> Блочное[Блочное хранение]
    Хранилища --> Витрины[Витрины данных, формат хранения]
    Хранилища --> Целостность[Целостность, состояние]
    Хранилища --> XD[Что такое XD?]
    Хранилища --> ПЛАК[ПЛАК]
    Хранилища --> Структура[Едина структура классификации информации в организации]

    ПЛАК --> DAS[DAS]
    ПЛАК --> NAS[NAS]
    ПЛАК --> SAN[SAN]

    Архитектура --> Файлы[файлы + индекс + язык запросов]
    Файлы --> Оптимизация[оптимизация производительности (модели данных + реализация). NoSQL.]
    Оптимизация --> NoSQL_2[NoSQL.]
  
```

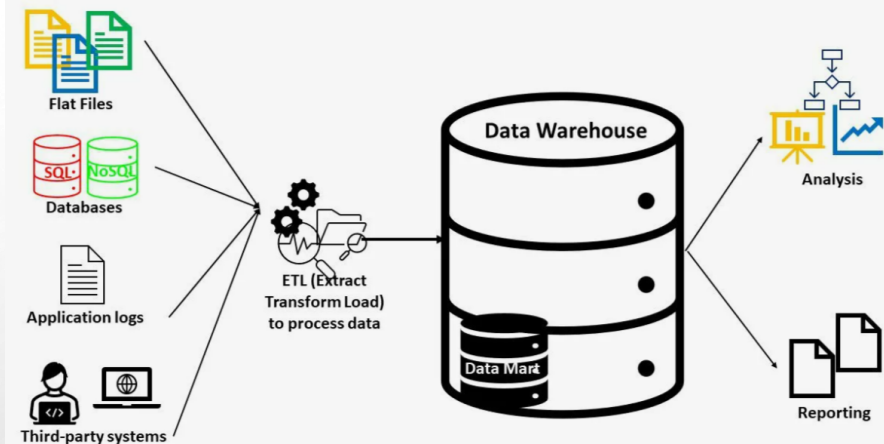


2 курс.

Проектирование баз данных

3 курс.

Разработка баз данных



Магистратура

Проектирование и разработка баз и хранилищ данных

Тема 7_ Оптимизация SQL-запросов. Транзакции

7.1_Транзакции	12
ACID	19
Изоляция транзакций	21
7.2. Индексы	45
7.3. Оптимизация запросов	69

Понятие транзакции

Транзакция - это неделимая последовательность операций манипулирования данными, переводящая базу данных из одного ***целостного состояния*** в другое целостное состояние.

Механизм транзакций:

- ✓ обеспечивает целостность БД,
- ✓ обеспечивает возможность восстановления БД после сбоев,
- ✓ обеспечивает изолированность пользователей в многопользовательских системах.

Операторы работы с транзакциями

1. **BEGIN** — начать блок транзакции. Синтаксис:

BEGIN [WORK | TRANSACTION] [режим_транзакции [, ...]]

где WORK | TRANSACTION - необязательные ключевые слова, не оказывают никакого влияния.

2. **COMMIT** — зафиксировать текущую транзакцию. Синтаксис:

COMMIT [WORK | TRANSACTION] [AND [NO] CHAIN]

COMMIT фиксирует текущую транзакцию. Все изменения, произведённые транзакцией, становятся видимыми для других и гарантированно сохраняются в случае сбоя.

3. **SAVEPOINT** — устанавливает новую точку сохранения в текущей транзакции. Синтаксис:

SAVEPOINT <имя_точки_сохранения>

Операторы работы с транзакциями

4. **ROLLBACK** — прервать текущую транзакцию. Синтаксис:

- **ROLLBACK [WORK | TRANSACTION] [AND [NO] CHAIN]**

ROLLBACK откатывает текущую транзакцию и приводит к аннулированию всех изменений, произведённых транзакцией.

- **ROLLBACK TO SAVEPOINT** — откатиться к точке сохранения. Синтаксис:

ROLLBACK [WORK | TRANSACTION] TO [SAVEPOINT] имя_точки_сохранения

Откатывает все команды, выполненные после установления точки сохранения, и начинает новую подтранзакцию на том же уровне транзакции.

5. **RELEASE SAVEPOINT** — высвободить ранее определённую точку сохранения. Синтаксис:

RELEASE [SAVEPOINT] имя_точки_сохранения

Виды транзакций

1. **Автоматическая транзакция** - каждая отдельная инструкция является транзакцией.

Предположим, что необходимо перевести 100 долларов с одного счета на другой. В упрощенном виде SQL-команды можно записать так:

Списание денег со счета клиента:

UPDATE accounts SET balance = balance - 100.00 WHERE name = 'Alice';

Зачисление денег на счет клиента:

UPDATE accounts SET balance = balance + 100.00 WHERE name = 'Bob';

Виды транзакций

2. Явная транзакция - каждая транзакция явно начинается BEGIN и явно заканчивается инструкцией COMMIT или ROLLBACK.

Банковская транзакция будет следующей:

BEGIN;

UPDATE accounts SET balance = balance - 100.00 WHERE name = 'Alice';

UPDATE accounts SET balance = balance + 100.00 WHERE name = 'Bob';

COMMIT;

Виды транзакций

3. Неявная транзакция - новая транзакция неявно начинается, когда предыдущая транзакция завершена, но каждая транзакция явно завершается инструкцией COMMIT или ROLLBACK

Для включения этого режима в СУБД PostgresPro необходимо выполнить команду:

SET implicit_transactions = ON;

Точки сохранения

Точки сохранения позволяют выборочно отменять некоторые части транзакции и фиксировать все остальные:

```
BEGIN;
```

```
UPDATE accounts SET balance = balance - 100.00 WHERE name = 'Alice';
```

```
SAVEPOINT my_savepoint;
```

```
UPDATE accounts SET balance = balance + 100.00 WHERE name = 'Bob';
```

```
-- ошибочное действие... забыть его и использовать счёт Уолли
```

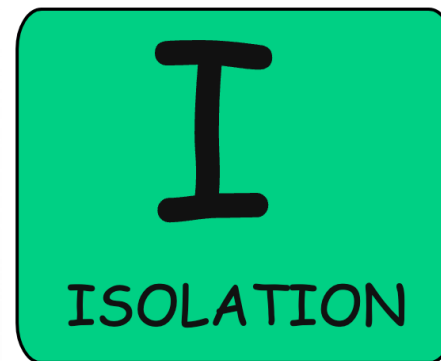
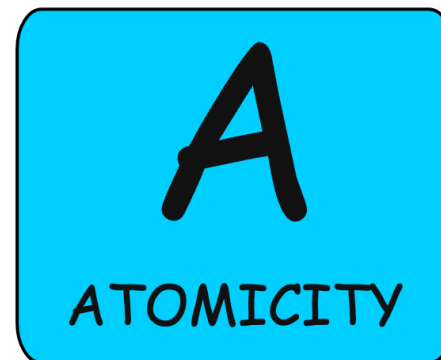
```
ROLLBACK TO my_savepoint;
```

```
UPDATE accounts SET balance = balance + 100.00 WHERE name = 'Wally';
```

```
COMMIT;
```

Свойства транзакций

ACID — набор требований к транзакционной системе, обеспечивающий наиболее надёжную и предсказуемую её работу — атомарность, согласованность, изоляцию, устойчивость; сформулированы в конце 1970-х годов Джимом Греем.



Свойства транзакций

1. **(A) Atomicity (Атомарность)** - Транзакция выполняется как атомарная операция - либо выполняется вся транзакция целиком, либо она целиком не выполняется (происходит откат транзакции)
2. **(C) Consistency (Согласованность)** - Транзакция переводит базу данных из одного согласованного (целостного) состояния в другое согласованное (целостное) состояние. Внутри транзакции согласованность базы данных может нарушаться.
3. **(I) Isolation (Изолированность)** - Во время выполнения транзакции параллельные транзакции не должны оказывать влияния на её результат.
4. **(D) Durability (Долговечность, Устойчивость)** - После успешного завершения транзакции, изменения, произведенные ею, надежно сохраняются и не могут быть утеряны даже в случае системных сбоев.

Изоляция транзакций

При параллельном выполнении транзакций возможны следующие аномалии (проблемы):

1. **«Грязное» чтение (dirty read)** — транзакция читает данные, записанные параллельной незавершённой транзакцией.
2. **Неповторяющееся чтение (non-repeatable read)** — это аномалия транзакций, при которой одна и та же строка данных, прочитанная дважды в рамках одной транзакции, имеет разные значения.
3. **Фантомное чтение (phantom reads)** — это проблема в SQL, когда повторный запрос в рамках одной транзакции возвращает другой набор строк, потому что другая транзакция добавила или удалила строки между выполнениями запросов.
4. **Аномалия сериализации** — это ситуация, когда результат параллельного выполнения транзакций не соответствует ни одному из возможных результатов их последовательного выполнения.

«Грязное» чтение

Предположим, имеются две транзакции, открытые различными приложениями, в которых выполнены следующие SQL-операторы:

Транзакция 1	Транзакция 2
UPDATE tbl1 SET f2=f2+1 WHERE f1=1;	
	SELECT f2 FROM tbl1 WHERE f1=1;
ROLLBACK;	

- В транзакции 1 изменяется значение поля f2,
- затем в транзакции 2 выбирается значение этого поля.
- После этого происходит откат транзакции 1.
- В результате значение, полученное второй транзакцией, будет отличаться от значения, хранимого в базе данных.

Неповторяющееся чтение

Предположим, имеются две транзакции, открытые различными приложениями, в которых выполнены следующие SQL-операторы:

Транзакция 1	Транзакция 2
	SELECT f2 FROM tbl1 WHERE f1=1;
UPDATE tbl1 SET f2=f2+3 WHERE f1=1;	
COMMIT;	
	SELECT f2 FROM tbl1 WHERE f1=1;

- В транзакции 2 выбирается значение поля f2,
- затем в транзакции 1 изменяется значение поля f2.
- При повторной попытке выбора значения из поля f2 в транзакции 2 будет получен другой результат.

Фантомное чтение

Предположим, имеется две транзакции, открытые различными приложениями, в которых выполнены следующие SQL-операторы:

Транзакция 1	Транзакция 2
	SELECT SUM(f2) FROM tbl1;
INSERT INTO tbl1 (f1,f2) VALUES (15,20);	
COMMIT;	
	SELECT SUM(f2) FROM tbl1;

- В транзакции 2 выполняется SQL-оператор, использующий все значения поля f2.
- Затем в транзакции 1 выполняется вставка новой строки, приводящая к тому, что повторное выполнение SQL-оператора в транзакции 2 выдаст другой результат.

Аномалия сериализации

Предположим, имеются две транзакции, выполняемые одновременно:

Транзакция 1	Транзакция 2
UPDATE tbl1 SET f2=f2+20 WHERE f1=1;	UPDATE tbl1 SET f2=f2+25 WHERE f1=1;

- Транзакции пытаются записать результат вычислений в поле f2. Поскольку одновременно две записи выполнить невозможно, в реальности одна из операций записи будет выполнена раньше, другая позже. При этом вторая операция записи перезапишет результат первой.
- В результате значение поля f2 по завершении обеих транзакций может увеличиться не на 45, а на 20 или 25, то есть одна из изменяющих данные транзакций «пропадёт».

Уровни изоляции транзакций стандарта SQL

Уровень изоляции транзакций - это степень обеспечиваемой внутренними механизмами СУБД защиты от всех или некоторых видов несогласованности данных, возникающих при параллельном выполнении транзакций.

Стандарт SQL-92 определяет четыре уровней изоляции:

- **Read uncommitted** - *уровень чтения незафиксированных данных*
- **Read committed** - *уровень чтения зафиксированных данных*
- **Repeatable read** - *уровень повторяющегося чтения*
- **Serializable** - *уровень способности к упорядочению*

В разных СУБД эти требования могут реализовываться разными способами (в частности, с использованием блокировок)

Поддержка изоляции транзакций в PostgresPro

- В Postgres Pro можно запросить любой из четырёх уровней изоляции транзакций. Однако внутри реализованы только три различных уровня, т.к. режим Read Uncommitted действует как Read Committed
- Реализация Repeatable Read в Postgres Pro не допускает фантомного чтения

Уровень изоляции	«Грязное» чтение	Неповторяемое чтение	Фантомное чтение	Аномалия сериализации
Read uncommitted (Чтение незафиксированных данных)	Допускается, но не в PG	Возможно	Возможно	Возможно
Read committed (Чтение зафиксированных данных)	Невозможно	Возможно	Возможно	Возможно
Repeatable read (Повторяемое чтение)	Невозможно	Невозможно	Допускается, но не в PG	Возможно
Serializable (Сериализуемость)	Невозможно	Невозможно	Невозможно	Невозможно

Выбор уровня изоляции транзакций

Для выбора нужного уровня изоляции транзакций используется команда **SET TRANSACTION:**

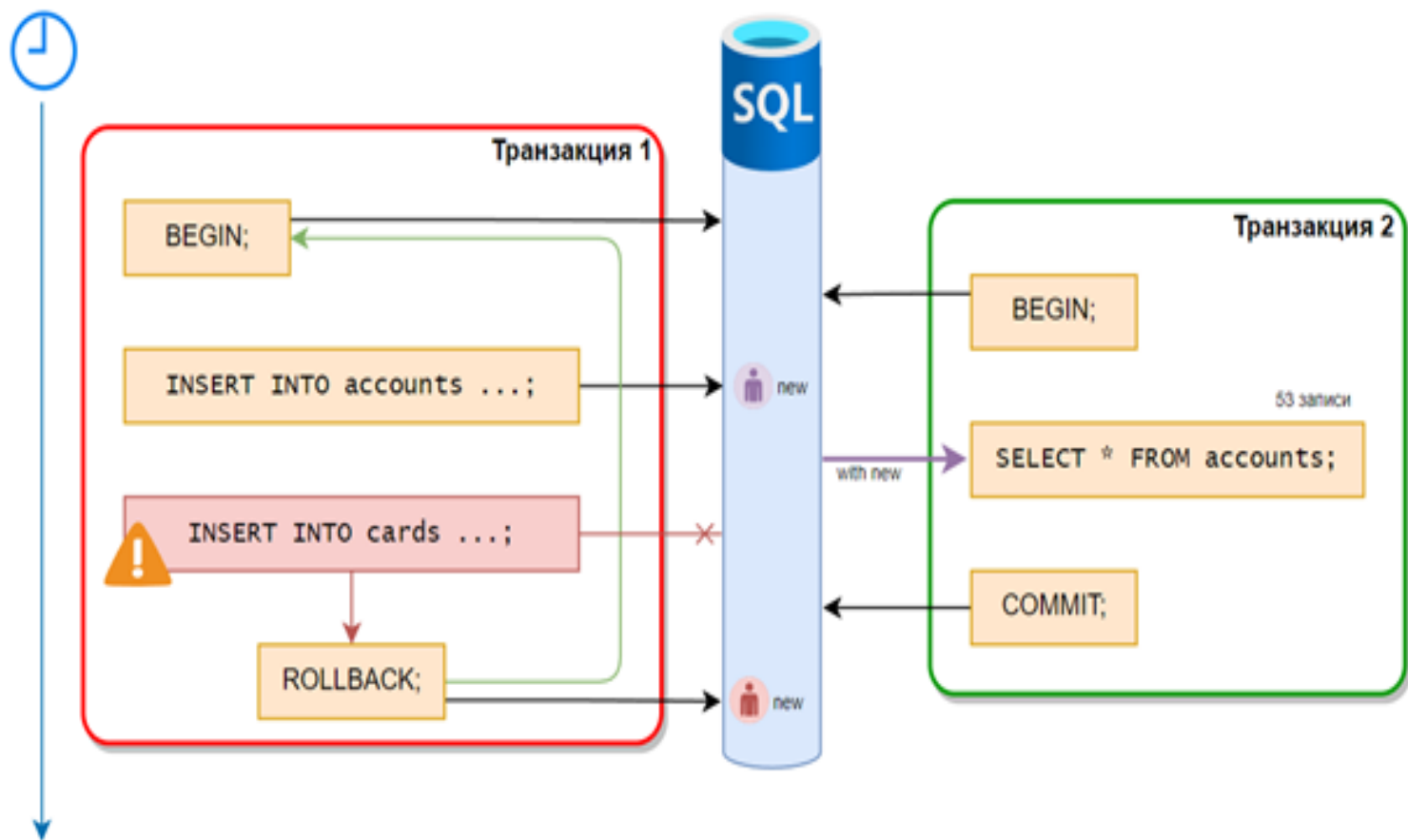
```
BEGIN;
```

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

```
...
```

```
COMMIT;
```

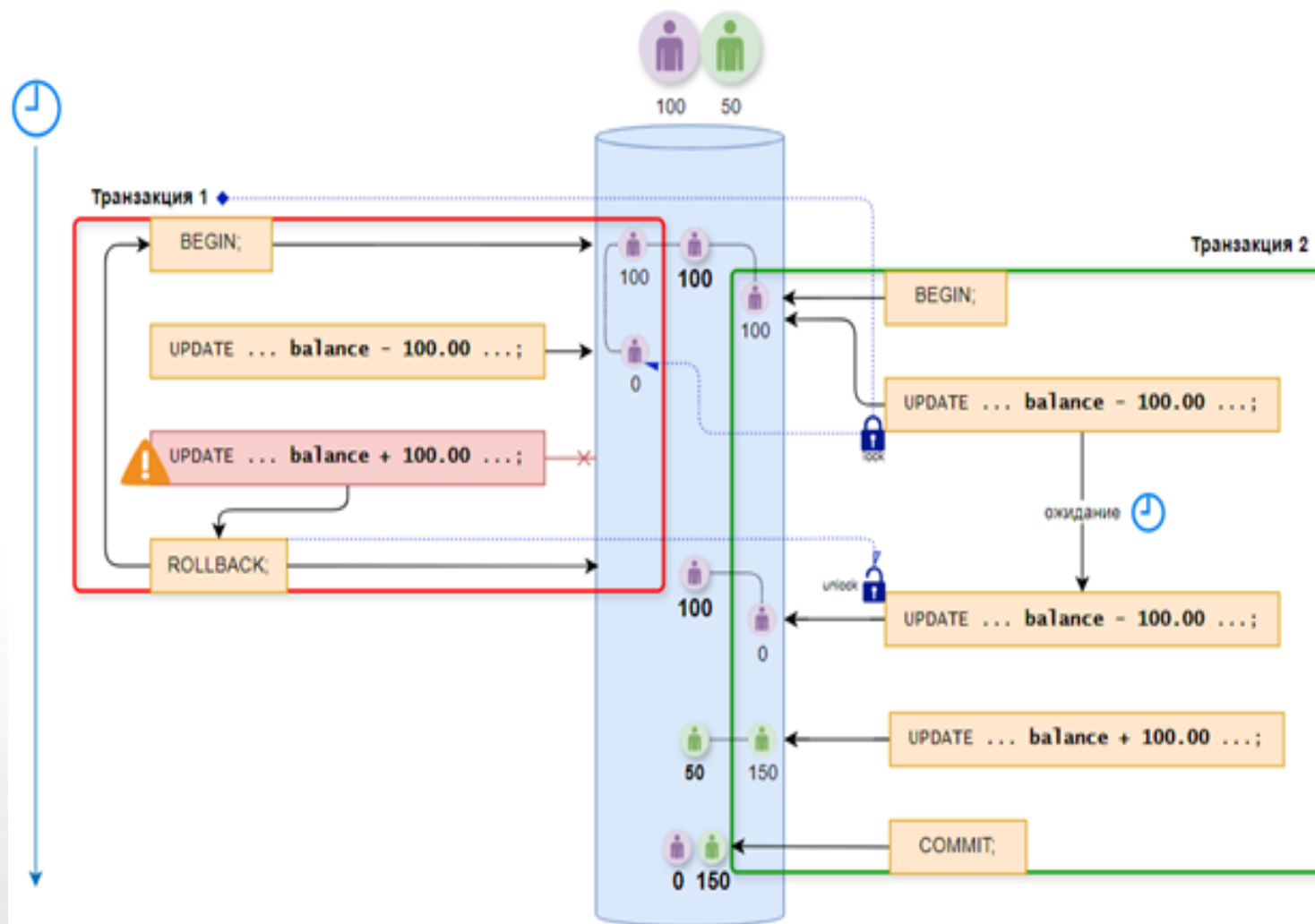

Read uncommitted



Низший уровень изоляции.

Уровень изоляции позволяет читать изменения, которые создала незавершенная транзакция: если несколько параллельных транзакций пытаются изменить одну и ту же строку таблицы, то в окончательном варианте строка будет иметь значение, определённое всем набором успешно выполненных транзакций.

Read committed

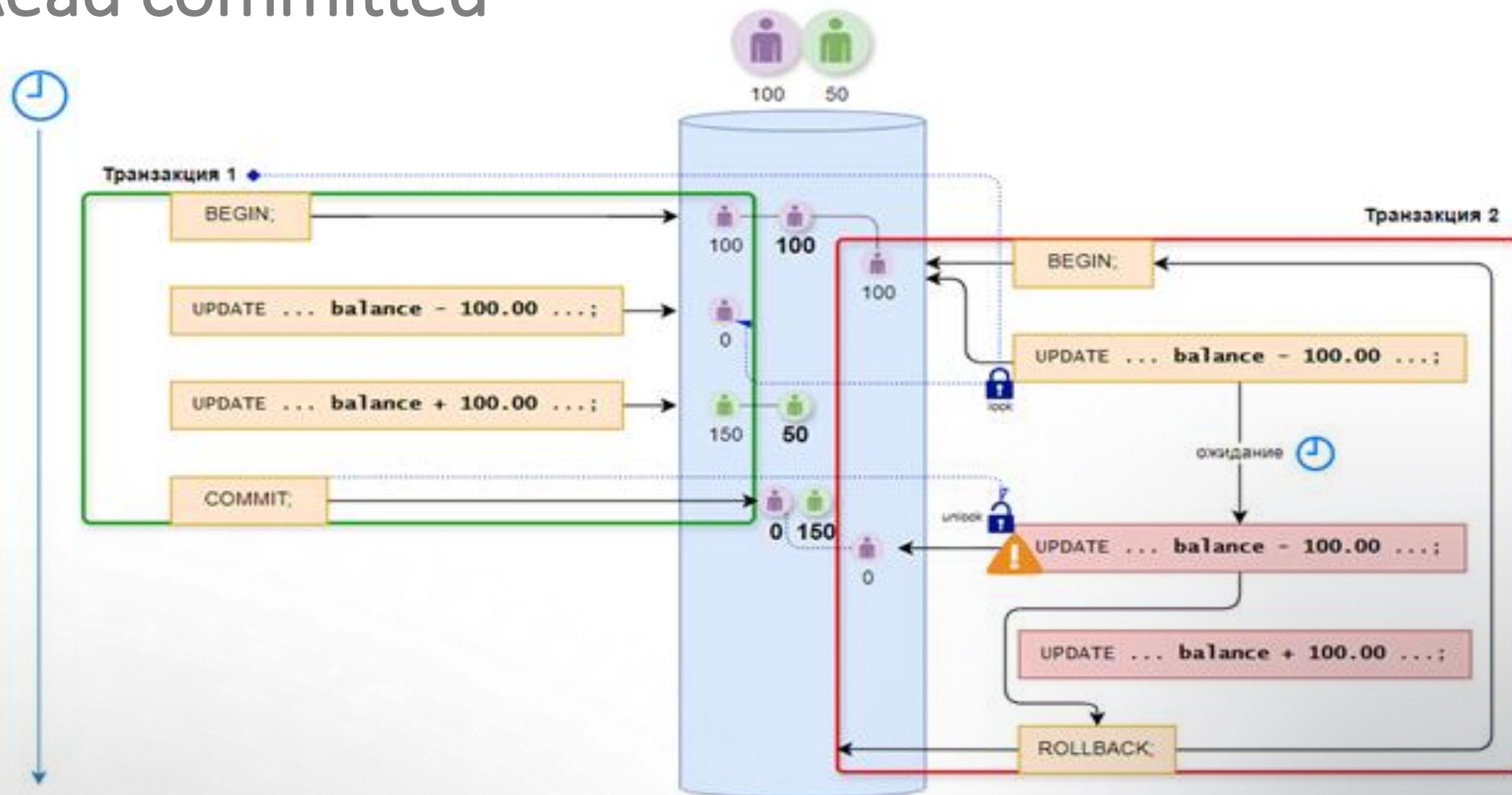


Основной принцип этого уровня – чтение только зафиксированных данных – исключает «грязное» чтение:

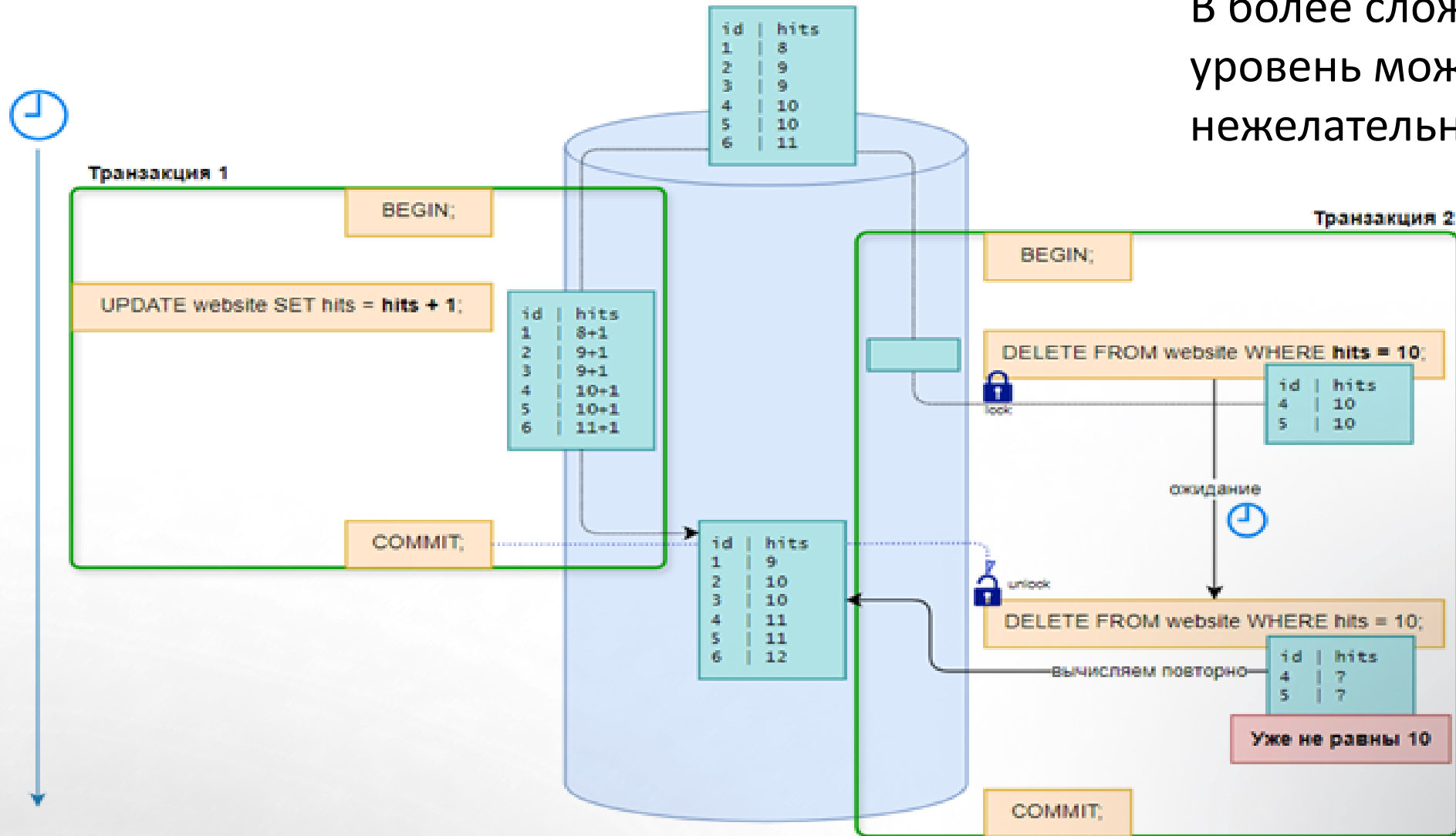
1. Каждый SELECT видит зафиксированную версию базы данных на момент своего старта.

2. Команды UPDATE, DELETE, SELECT FOR UPDATE и SELECT FOR SHARE видят зафиксированные строки, но запланированные изменения могут быть отложено до фиксирования или отката первой изменяющей данные транзакции (если она ещё выполняется)

Read committed



Read committed

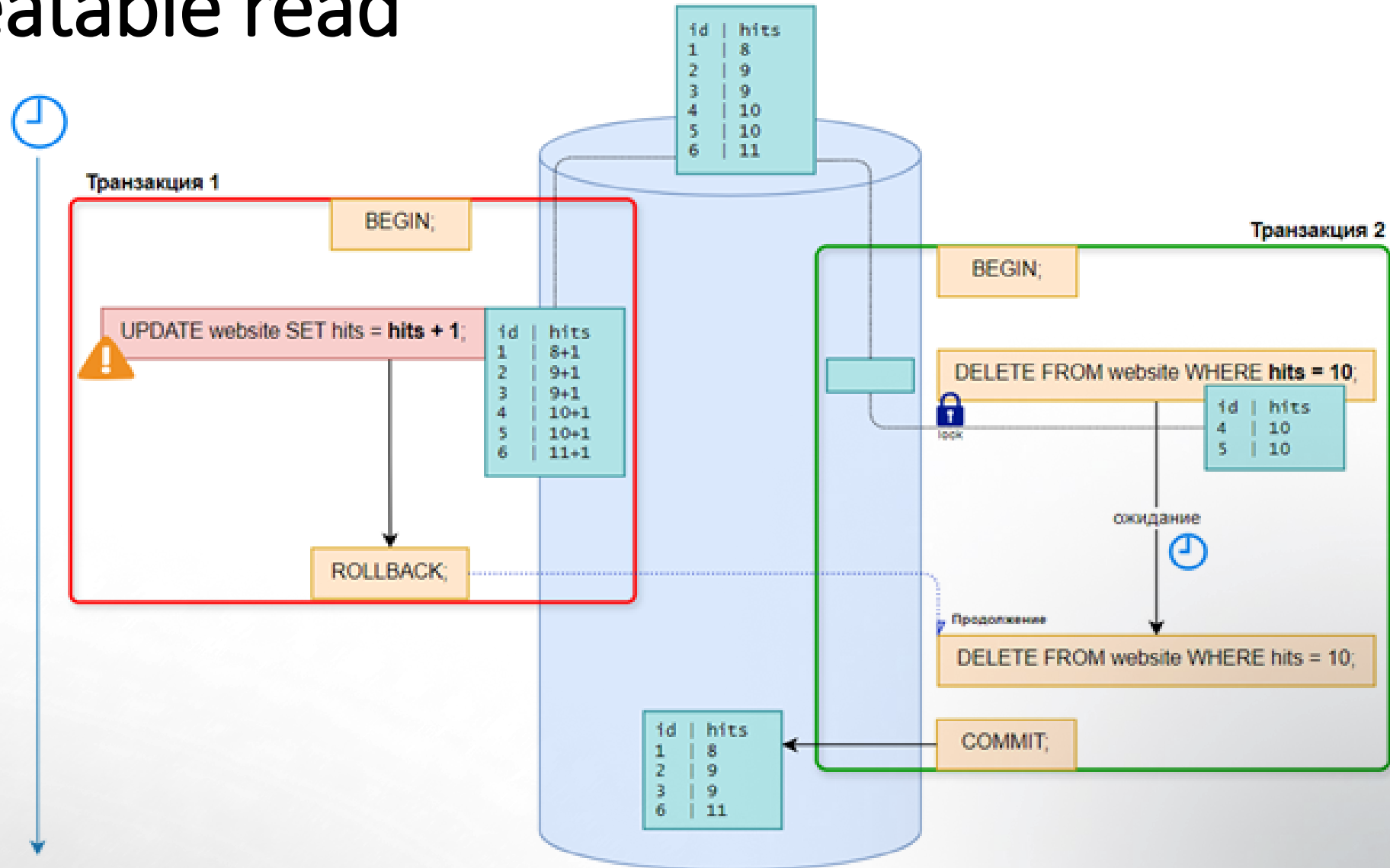


В более сложных ситуациях этот уровень может приводить к нежелательным результатам.

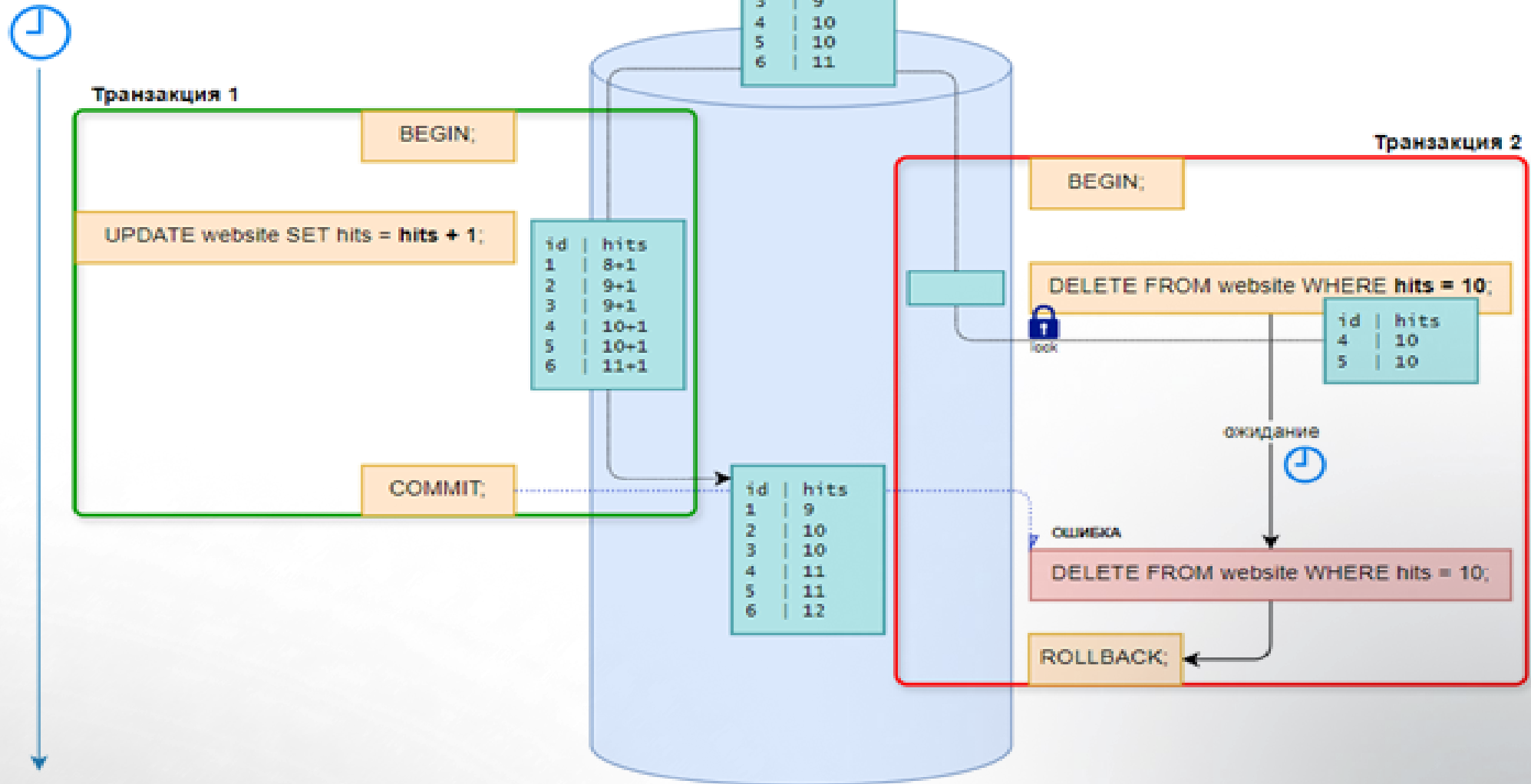
Repeatable read

- В режиме Repeatable Read видны только те данные, которые были зафиксированы до начала транзакции, но не видны незафиксированные данные и изменения, произведённые другими транзакциями в процессе выполнения данной.
- Этот уровень отличается от Read Committed тем, что запрос в транзакции данного уровня видит снимок данных на момент начала первого оператора в транзакции , а не начала текущего оператора:
 - Команды SELECT в одной транзакции **видят одни и те же данные**. Они не видят изменений, внесённых и зафиксированных другими транзакциями после начала их текущей транзакции.
 - Команды UPDATE, DELETE, MERGE, SELECT FOR UPDATE и SELECT FOR SHARE найдут только те целевые строки, которые были зафиксированы **на момент начала транзакции**.

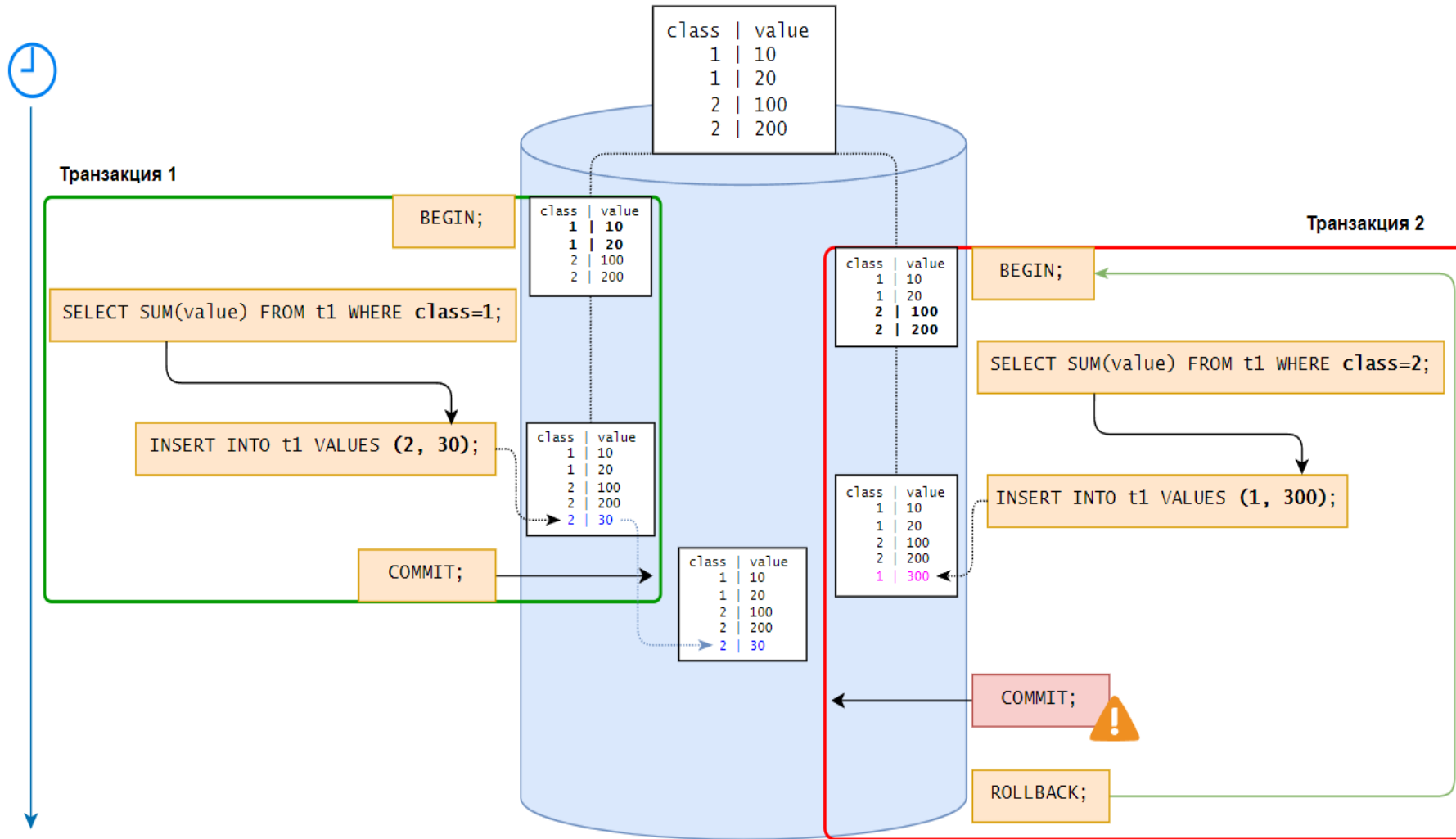
Repeatable read



Repeatable read



Serializable



Самый высокий уровень изолированности; транзакции полностью изолируются друг от друга. Результат выполнения нескольких параллельных транзакций должен быть таким, как если бы они выполнялись последовательно.

Сериализация транзакций

Для выполнения требования изолированности транзакций, в СУБД должны использоваться какие-либо методы регулирования совместного выполнения транзакций.

Пусть в системе одновременно выполняется некоторое множество транзакций $S = \{T_1, T_2, \dots, T_n\}$. Транзакции называются **конкурирующими**, если они пересекаются по времени и обращаются к одним и тем же данным.

Сериальный план – это план выполнения набора транзакций S , в котором, $T_1, T_2, \dots, T_n$ чередуются или реально параллельно выполняются операции разных транзакций $T_1, T_2, \dots, T_n$ и при этом результат совместного выполнения транзакций эквивалентен результату некоторого последовательного выполнения этих же транзакций

Сериализация транзакций – это механизм одновременного выполнения транзакций $T_1, T_2, \dots, T_n$ по некоторому сериальному плану.

Обеспечение такого механизма является основной функцией компонента СУБД, ответственного за управление транзакциями.

Конфликты доступа к данным

Виды конфликтов *доступа* к данным:

- ✓ **W-W (Запись - Запись).** Первая транзакция изменила объект и не закончилась. Вторая транзакция пытается изменить этот объект. Результат - потеря обновления.
- ✓ **R-W (Чтение - Запись).** Первая транзакция прочитала объект и не закончилась. Вторая транзакция пытается изменить этот объект. Результат - неповторяющиеся чтения.
- ✓ **W-R (Запись - Чтение).** Первая транзакция изменила объект и не закончилась. Вторая транзакция пытается прочитать этот объект. Результат - чтение "грязных" данных.

Блокировки

Блокировки — это механизм, предотвращающий конфликты при параллельном доступе к данным, гарантируя целостность и согласованность данных.

Типы блокировок:

- ✓ **Исключительные блокировки** (**X-блокировки**, **X-locks** - eXclusive locks) - блокировки без взаимного доступа (блокировка записи).
- ✓ **Разделяемые блокировки** (**S-блокировки**, **S-locks** - Shared locks) - блокировки с взаимным доступом (блокировка чтения).

Уровни блокировок:

- Блокировки на уровне строк
- Блокировки на уровне таблиц
- Блокировки на уровне страниц
- Блокировки объектов

Просмотреть текущие блокировки можно с помощью представления pg_locks.

Хотим заблокировать		
	X	S
-	✓	✓
X	✗	✗
S	✗	✓

Текущее состояние объекта

Блокировки на уровне строк

Блокировки на уровне строк блокируют только запись в определённые строки, но никак не влияют на выборку.

Режимы явной блокировки на уровне строк:

- **FOR UPDATE**

В режиме FOR UPDATE строки, выданные оператором SELECT, блокируются как для изменения.

- **FOR NO KEY UPDATE**

Действует подобно FOR UPDATE, но запрашиваемая в этом режиме блокировка слабее: она не будет блокировать команды SELECT FOR KEY SHARE, пытающиеся получить блокировку тех же строк.

- **FOR SHARE**

Действует подобно FOR NO KEY UPDATE, за исключением того, что для каждой из полученных строк запрашивается разделяемая, а не исключительная блокировка.

- **FOR KEY SHARE**

Действует подобно FOR SHARE, но устанавливает более слабую блокировку: блокируется SELECT FOR UPDATE, но не SELECT FOR NO KEY UPDATE.

Конфликты блокировок на уровне строк

Запрашиваемый режим блокировки	Текущий режим блокировки			
	FOR KEY SHARE	FOR SHARE	FOR NO KEY UPDATE	FOR UPDATE
FOR KEY SHARE				X
FOR SHARE			X	X
FOR NO KEY UPDATE		X	X	X
FOR UPDATE	X	X	X	X

Блокировки на уровне таблиц

`LOCK [ TABLE ] [ ONLY ] name [ * ] [, ...] [ IN lockmode MODE ] [ NOWAIT ]`

где **lockmode** один из режимов:

- **ACCESS SHARE** - накладываются запросами, которые только читают данные из таблицы, но не модифицируют ее. Обычно это запрос SELECT.
- **ROW SHARE** - накладываются запросами SELECT FOR UPDATE и SELECT FOR SHARE.
- **ROW EXCLUSIVE** - накладываются запросами, которые модифицируют данные в таблице. Обычно это запросы UPDATE, DELETE и INSERT.
- **SHARE UPDATE EXCLUSIVE** - накладывается VACUUM, параллельными индексами, статистикой и некоторыми вариантами команд ALTER TABLE. Этот режим защищает от параллельных изменений схемы и выполнения вакуумации.

Блокировки на уровне таблиц

- **SHARE** - накладывается командой `CREATE INDEX`, которая не выполняется в параллельном режиме. Этот режим защищает таблицу от параллельного изменения данных.
- **SHARE ROW EXCLUSIVE** - Этот режим защищает таблицу от параллельного изменения данных и является полуэксклюзивным, так что только одна сессия может удерживать ее в каждый момент времени. Накладывается операторами `CREATE COLLATION`, `CREATE TRIGGER` и многими формами оператора `ALTER TABLE`.
- **EXCLUSIVE** - Этот режим допускает только параллельные блокировки `ACCESS SHARE`, т.е. только чтение из таблицы может происходить параллельно с транзакцией, удерживающий этот режим блокировки.
- **ACCESS EXCLUSIVE** - этот режим гарантирует, что удерживает блокировку только транзакция, получившая доступ к таблице. Накладывается командами `DROP TABLE`, `ALTER TABLE`, `VACUUM FULL`.

Конфликты блокировок на уровне таблиц

Запрашиваемый режим блокировки	Существующий режим блокировки							
	ACCESS SHARE	ROW SHARE	ROW EXCL.	SHARE UPDATE EXCL.	SHARE	SHARE ROW EXCL.	EXCL.	ACCESS EXCL.
ACCESS SHARE								X
ROW SHARE							X	X
ROW EXCL.					X	X	X	X
SHARE UPDATE EXCL.				X	X	X	X	X
SHARE			X	X		X	X	X
SHARE ROW EXCL.			X	X	X	X	X	X
EXCL.		X	X	X	X	X	X	X
ACCESS EXCL.	X	X	X	X	X	X	X	X

Индексы. Основные понятия

Индексы — это традиционное средство увеличения производительности БД. Используя индекс, сервер баз данных может находить и извлекать необходимые строки гораздо быстрее, чем без него.

Общий синтаксис этой команды в документации к PostgreSQL:

CREATE [UNIQUE] INDEX [CONCURRENTLY] [[IF NOT EXISTS] имя]

ON [ONLY] имя_таблицы [USING метод] ({ имя_столбца | (выражение) } [COLLATE правило_сортировки] [класс_операторов [(параметр_класса_оп = значение [, ...])]] [ASC | DESC] [NULLS { FIRST | LAST }] [, ...])

[INCLUDE (имя_столбца [, ...])]

[NULLS [NOT] DISTINCT]

[WITH (параметр_хранения [= значение] [, ...])]

[TABLESPACE табл_пространство]

[WHERE предикат]

Индексы. Основные понятия

Предположим, что у нас есть такая таблица:

```
CREATE TABLE test1 (  
    id integer,  
    count integer,  
    content varchar(100)  
);
```

и приложение выполняет много подобных запросов:

```
SELECT content FROM test1 WHERE count =  
константа;
```

Создать индекс:

```
CREATE INDEX test1_count_index ON test1 (count);
```

Предметный указатель

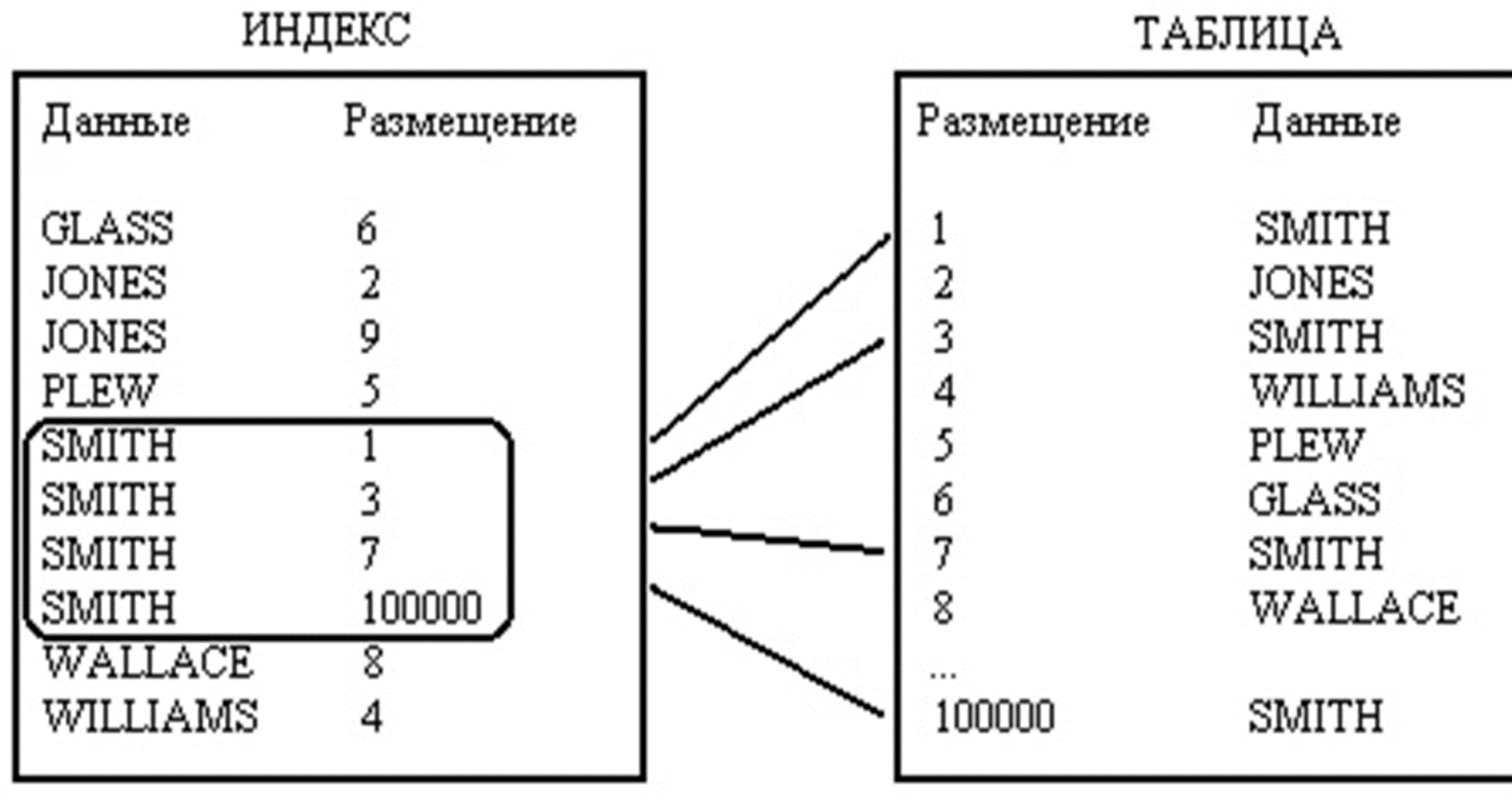
А

Абсолют	36
Абсцисса	81
Автоколебания	175
Аксиальный вектор	67
Алгебра мер	279

Б

Борелевская функция	11
Брауэра теорема	348
Брахистохрона	73
Бэра множество	245
Бюффона задача	154

Индексы. Основные понятия



Типы индексов

- **B-дерево**
- **Хеш**
- **GiST**
- **SP-GiST**
- **GIN**
- **BRIN**

По умолчанию команда `CREATE INDEX` создаёт индексы-B-деревья, эффективные в большинстве случаев.

Выбрать другой тип можно, написав название типа индекса после ключевого слова `USING`.

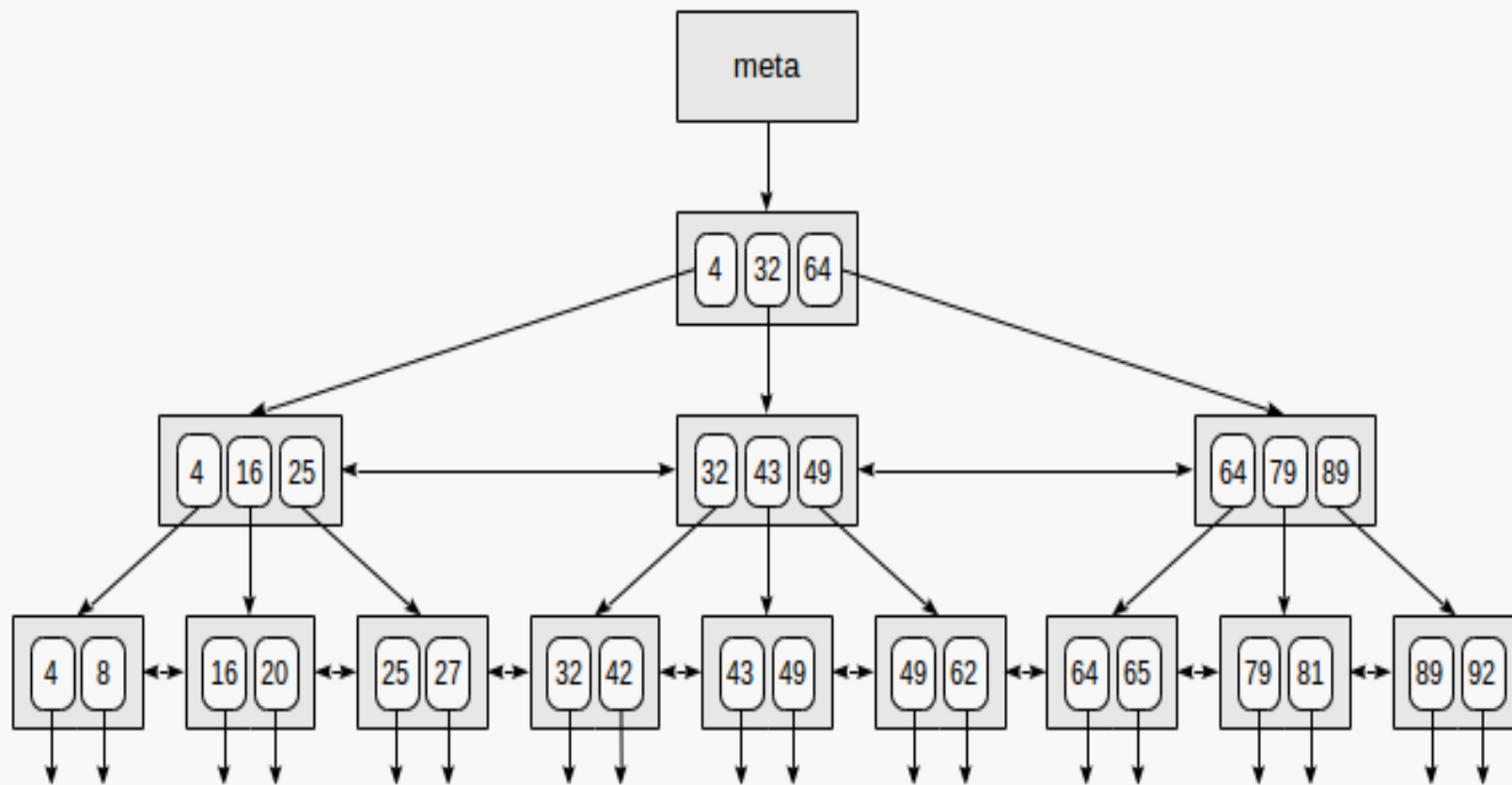
B-tree индекс

B-tree (Balanced Tree, Сбалансированное дерево) — это древовидная структура данных, которая сохраняет данные отсортированными и позволяет выполнять операции поиска, последовательного доступа, вставки и удаления за логарифмическое время. Каждое обновление дерева (вставка, обновление или удаление) приводит к его балансировке.

В-деревья обладают несколькими важными характеристиками:

- В-деревья сбалансированы, то есть каждая листовая страница отделена от корня одинаковым количеством внутренних страниц. Поэтому поиск любого значения занимает одинаковое время.
- В-деревья имеют много ветвей, то есть каждая страница (обычно 8 КБ) содержит множество (сотни) TID. В результате глубина В-деревьев довольно мала, до 4–5 для очень больших таблиц.
- Данные в индексе отсортированы по неубывающему порядку (как между страницами, так и внутри каждой страницы), а страницы одного уровня соединены друг с другом двунаправленным списком.

B-tree индекс



B-tree индекс

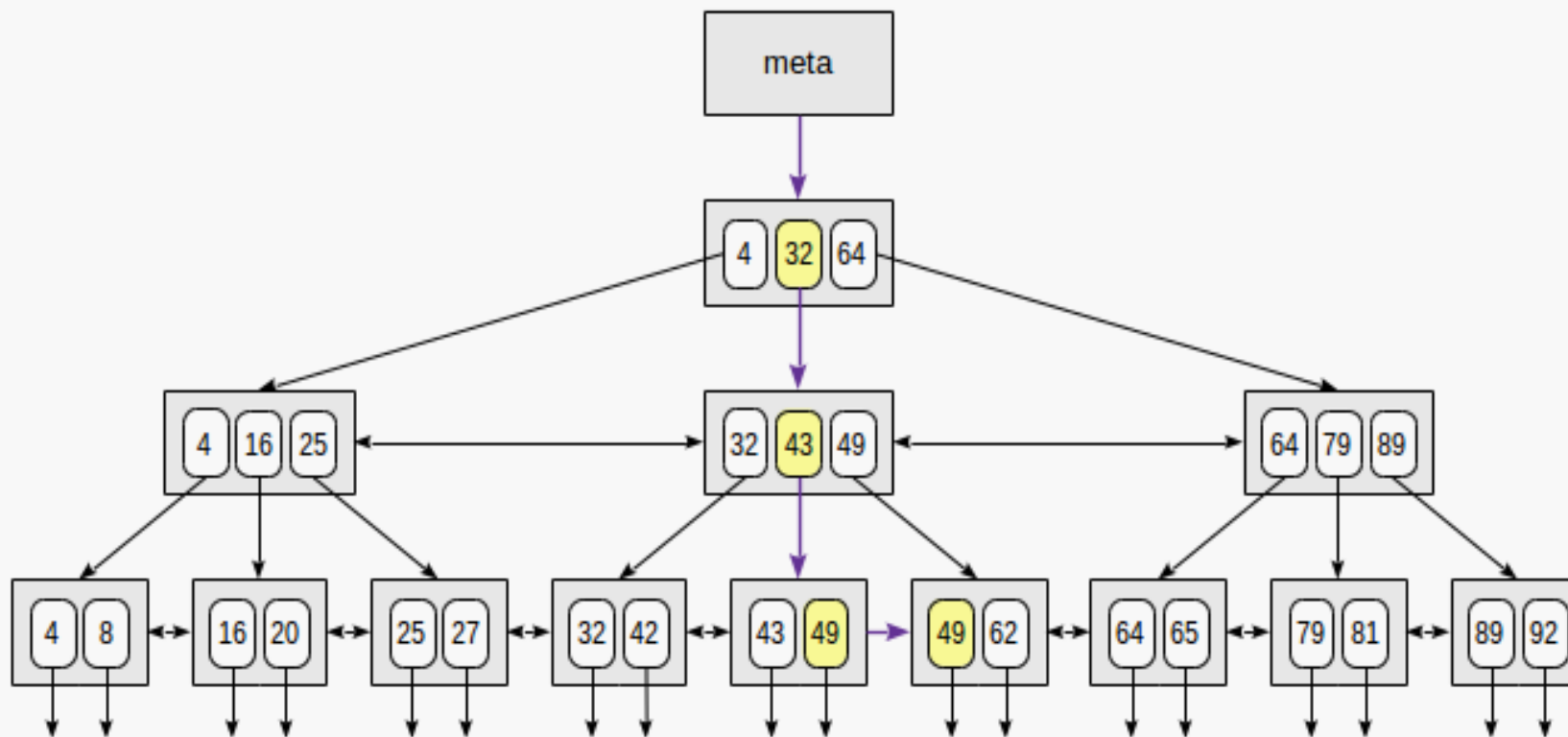
Строки индекса B-дерева упакованы в страницы:

- Самая первая страница индекса — это метастраница, ссылающаяся на корень индекса.
- На внутренних страницах каждая строка ссылается на дочернюю страницу индекса и содержит минимальное значение на этой странице.
- Листовые узлы — это узлы самого нижнего уровня, которые содержат **ключи индекса** (фактические значения из индексируемых столбцов) и **TID (Tuple ID)** - указатели на физическое расположение строки в таблице. TID состоит из номера блока и позиции строки внутри блока.

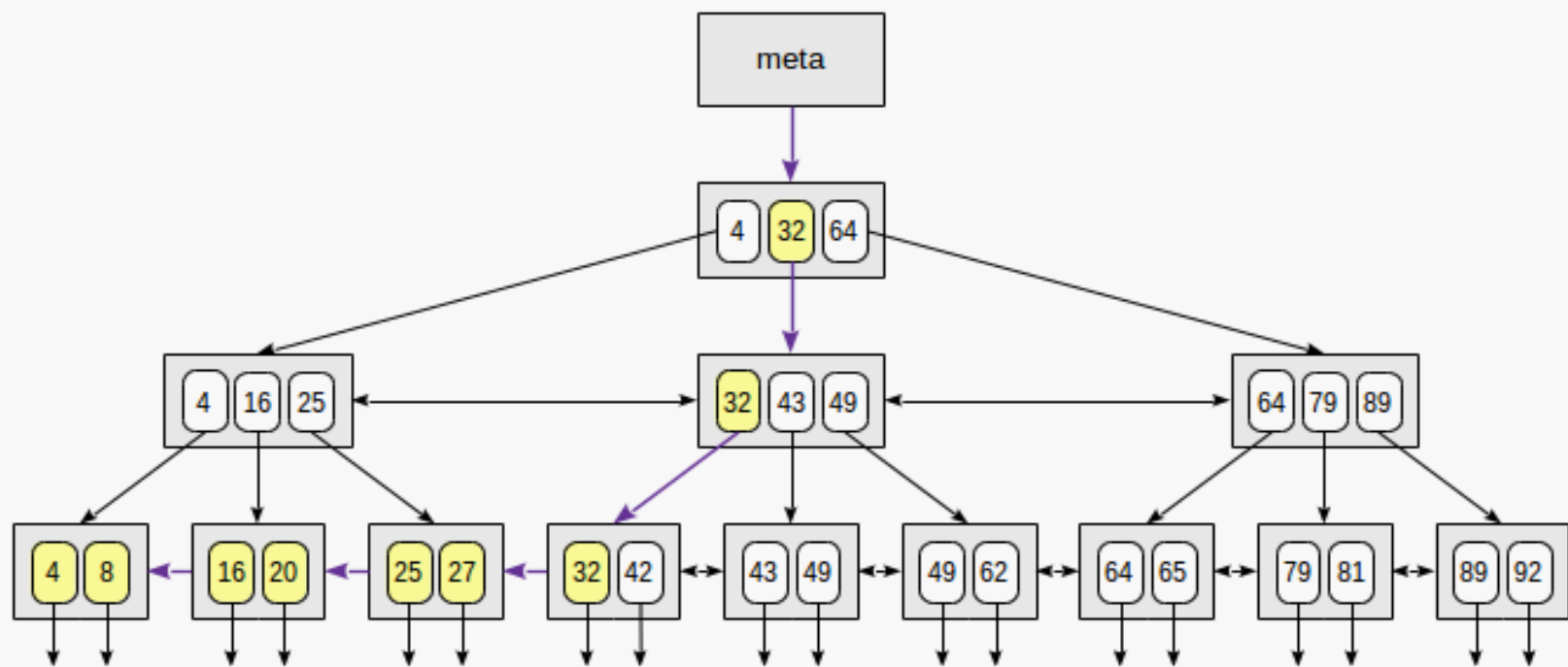
Свойства индексов со структурой B-Tree

- Количество операций ввода-вывода, необходимых для получения идентификатора строки, зависит от числа уровней ветвления дерева.
- При увеличении *индекса* в результате добавления новых данных СУБД добавляет в него новые уровни, чтобы обеспечить сбалансированность дерева (обычно не более 4-х).
- Значения в *индексе* упорядочиваются по ключу, а физические страницы *индекса* организуются в двунаправленный список.
- Обслуживание индексов происходит автоматически при любом изменении данных в таблице.

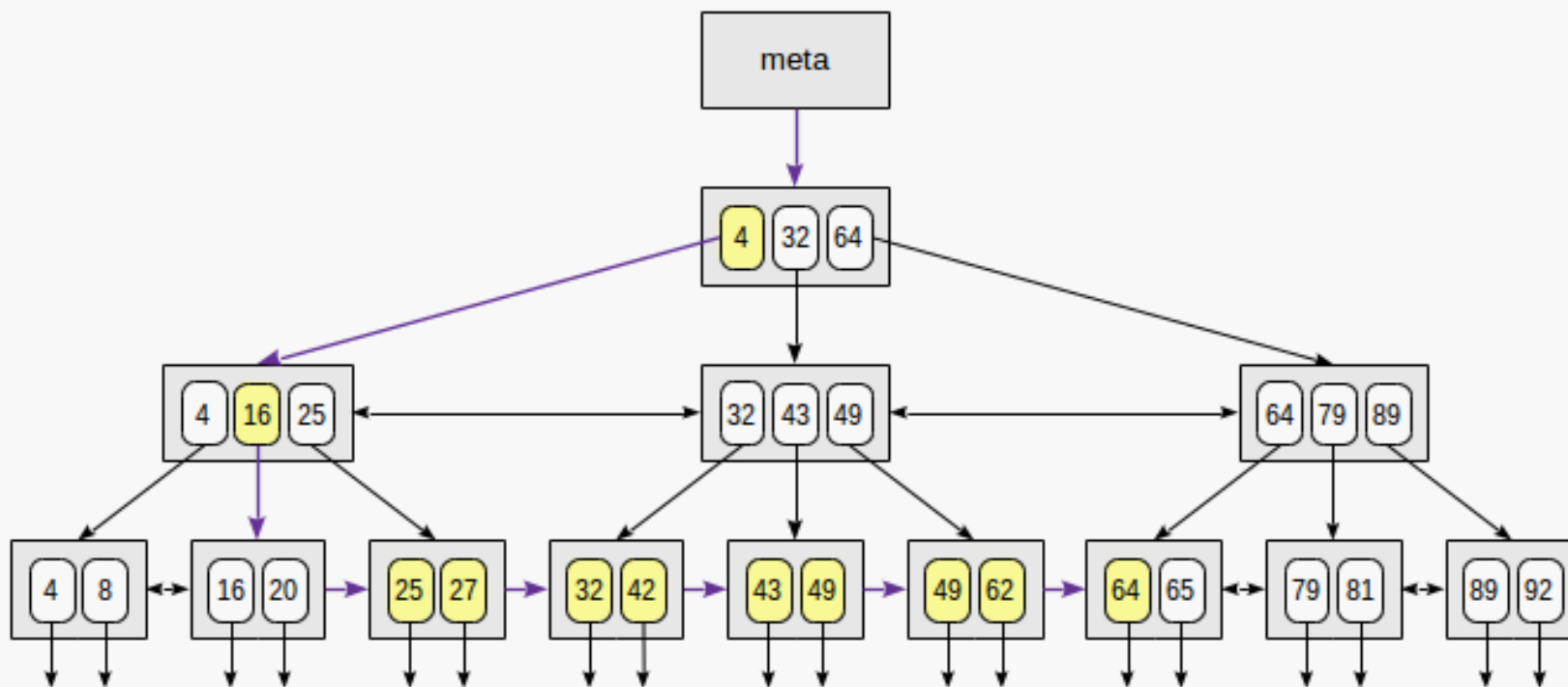
Поиск значения в дереве по условию равенства



Поиск значения в дереве по условию



Поиск значения в дереве по условию



Хеш-индекс

Хеш-индексы хранят 32-битный хеш-код, полученный из значения индексируемого столбца.

Поэтому хеш-индексы работают только если индексируемый столбец участвует в сравнении с оператором равенства.

Создать такой индекс можно командой:

CREATE INDEX имя ON таблица USING HASH (столбец);

GiST-индекс

GiST (Generalized Search Tree, Обобщенное поисковое дерево) - индексы представляют собой инфраструктуру, позволяющую реализовать много разных стратегий индексирования.

Подходят для использования со следующими типами: геометрические данные (point, box, lseg, line, path, polygon и circle), массивы, иерархические данные и для полнотекстового поиска.

Создать такой индекс можно командой:

```
CREATE INDEX locations_coordinates_gist_idx  
ON locations USING GIST (coordinates);
```

SP- GiST-индекс

SP-GiST (Space-Partitioned Generalized Search Tree, Пространственно-разделяемое обобщённое поисковое дерево) позволяет организовать на диске несбалансированные структуры данных, такие как деревья квадрантов, k-мерные и префиксные деревья. Такой индекс необходимо использовать для ускорения поиска при работе с IP-адресами, путями в деревьях или точками на плоскости и других типов пространственных данных.

Создать такой индекс можно командой:

```
CREATE INDEX idx_access_log_ip ON access_log USING SPGIST (ip_address);
```

Применение: **SELECT * FROM access_log WHERE ip_address << '192.168.1.0/24'::inet;**

GIN -индекс

GIN (Generalized Inverted Index) -индексы представляют собой «инвертированные индексы», в которых могут содержаться значения с несколькими ключами, например массивы. Инвертированный индекс содержит отдельный элемент для значения каждого компонента, и может эффективно работать в запросах, проверяющих присутствие определённых значений компонентов, таких как массивы, JSONB и полнотекстовые данные.

Создать такой индекс можно командой:

```
CREATE INDEX idx_profiles_data ON user_profiles USING GIN (profile);
```

Применение: **SELECT * FROM user_profiles WHERE profile ? 'email';**

BRIN -индекс

BRIN (Block Range Index)— это особый тип индекса в PostgreSQL, хранящий обобщённые сведения о значениях, находящихся в физически последовательно расположенных блоках таблицы. Поэтому такие индексы наиболее эффективны для столбцов, значения в которых хорошо коррелируют с физическим порядком столбцов таблицы, например для индексации временных данных.

Создать такой индекс можно командой:

```
CREATE INDEX idx_logs_time ON server_logs USING BRIN (log_time);
```

Применение:

```
SELECT * FROM server_logs  
WHERE log_time BETWEEN '2024-01-01' AND '2024-01-02';
```

Составной индекс, использование при сортировке

Для ускорения поиска по нескольким столбцам можно создать составной индекс.

Пример: Индекс, ускоряющий поиск одновременно по столбцам city и surname:

CREATE INDEX users_cs_idx ON test2 (city, surname);

По умолчанию элементы B-дерева хранятся в порядке возрастания, при этом значения NULL идут в конце. Можно изменить порядок сортировки элементов B-дерева, добавив уточнения ASC, DESC, NULLS FIRST и/или NULLS LAST при создании индекса.

Пример 1. Индекс будет сортирован по возрастанию, но NULL – значения – в начале дерева: **CREATE INDEX test2_info_nulls_low ON test2 (info NULLS FIRST);**

Пример 2. Создание индекса для сортировки по двум столбцам:

CREATE INDEX products_sort_idx ON products (category ASC, price DESC);

Уникальные индексы, индексы по выражениям

Синтаксис создания уникального индекса:

CREATE UNIQUE INDEX имя ON таблица (столбец [, ...]) [NULLS [NOT] DISTINCT];

По умолчанию значения NULL в уникальном столбце считаются не равными друг другу. Параметр **NULLS NOT DISTINCT** изменяет это правило

Пример. Создать уникальный индекс по столбцу title:

CREATE UNIQUE INDEX title_idx ON films (title);

Индекс можно создать не только по столбцу таблицы, но и по функции или скалярному выражению с одним или несколькими столбцами таблицы.

Пример. Для сравнений строк без учёта регистра символов часто используется функция lower:

CREATE INDEX test1_lower_col1_idx ON test1 (lower(col1));

Частичный индекс

Частичный индекс — это индекс, который строится по подмножеству строк таблицы, определяемому условным выражением (оно называется *предикатом* частичного индекса). Частичные индексы могут быть полезны, тем, что позволяют избежать индексирования распространённых значений.

Пример 1. Индекс только для незавершенных заказов:

```
CREATE INDEX idx_orders_pending  
    ON orders (user_id, created_at)  
    WHERE status IN ('pending', 'processing');
```

Пример 2. Индекс для записей за последний год:

```
CREATE INDEX idx_recent_orders  
    ON orders (created_at)  
    WHERE created_at >= CURRENT_DATE - INTERVAL '1 year';
```

Покрывающие индексы

Покрывающие индексы - это индексы, которые содержат помимо ключевых столбцов не ключевые столбцы. Это позволяет получить все необходимые данные прямо из индекса, не обращаясь к основной таблице.

**Пример. CREATE INDEX idx_employees_dept ON employees(department);
CREATE INDEX idx_employees_dept_covering ON employees(department) INCLUDE (name, salary);**

Рекомендуется:

- Частых запросов с фильтрацией и малым набором столбцов
- Столбцов, которые не участвуют в поиске, но часто запрашиваются в результирующем наборе
- Больших столбцов, которые не должны быть в ключе, например, содержащие json-объекты.

НЕ рекомендуется:

- Если включенные столбцы часто обновляются, в этом случае индекс часто перестраивается
- Когда необходима сортировка по включенным столбцам рекомендуется использовать составной индекс

Управление индексами

1. Удаление индекса

Синтаксис: **DROP INDEX [CONCURRENTLY] [IF EXISTS] имя [, ...] [CASCADE | RESTRICT]**

Пример: **DROP INDEX test1_count_index;**

2. Реорганизация (пересоздание) индекса

Синтаксис: **REINDEX { INDEX | TABLE | SCHEMA | DATABASE | SYSTEM } name;**

Пример: **REINDEX INDEX idx_users_email;**

- СУБД автоматически поддерживает состояние индексов при выполнении операций вставки, обновления или удаления.
- Со временем эти изменения могут привести к тому, что данные в индексе окажутся фрагментированными.
- Значительно фрагментированные индексы могут серьезно снижать производительность запросов.
- Можно устранить фрагментацию путем реорганизации индекса.

Селективность

- **Селективность (избирательность) индекса:** отношение количества уникальных значений к общему числу строк в индексе.

$$\text{Sel}_I = \text{DISTINCT_KEY} / \text{NUM_ROWS}$$

При значениях селективности **близких к 0** создание индекса неэффективно.

- **Кардинальность индекса:** количество уникальных значений в индексе.

$$\text{Car}_I = \text{DISTINCT_KEY}$$

При малом количестве уникальных значений создание индекса не эффективно

- **Селективность предиката поиска** (селективность таблицы): при равномерном распределении данных - это отношение числа строк, соответствующих конкретному ключевому значению, к общему числу строк в индексе (доля от общего количества строк, возвращаемых на одно значение предиката).

$$\text{Sel} = (\text{NUM_ROWS} / \text{DISTINCT_KEY}) / \text{NUM_ROWS} = 1 / \text{DISTINCT_KEY}$$

при селективности таблицы большей 0.33 создание индекса малоэффективно.

Рекомендации по созданию индексов

- Для первичных ключей и ограничений уникальности индексы создаются автоматически
- Создать для:
 - ✓ внешних ключей,
 - ✓ столбцов, участвующих в операции соединения,
 - ✓ столбцов с проверяемыми значениями,
 - ✓ столбцов, участвующих в агрегированных операциях
 - ✓ столбцов, участвующих в сортировке

Рекомендации по созданию индексов

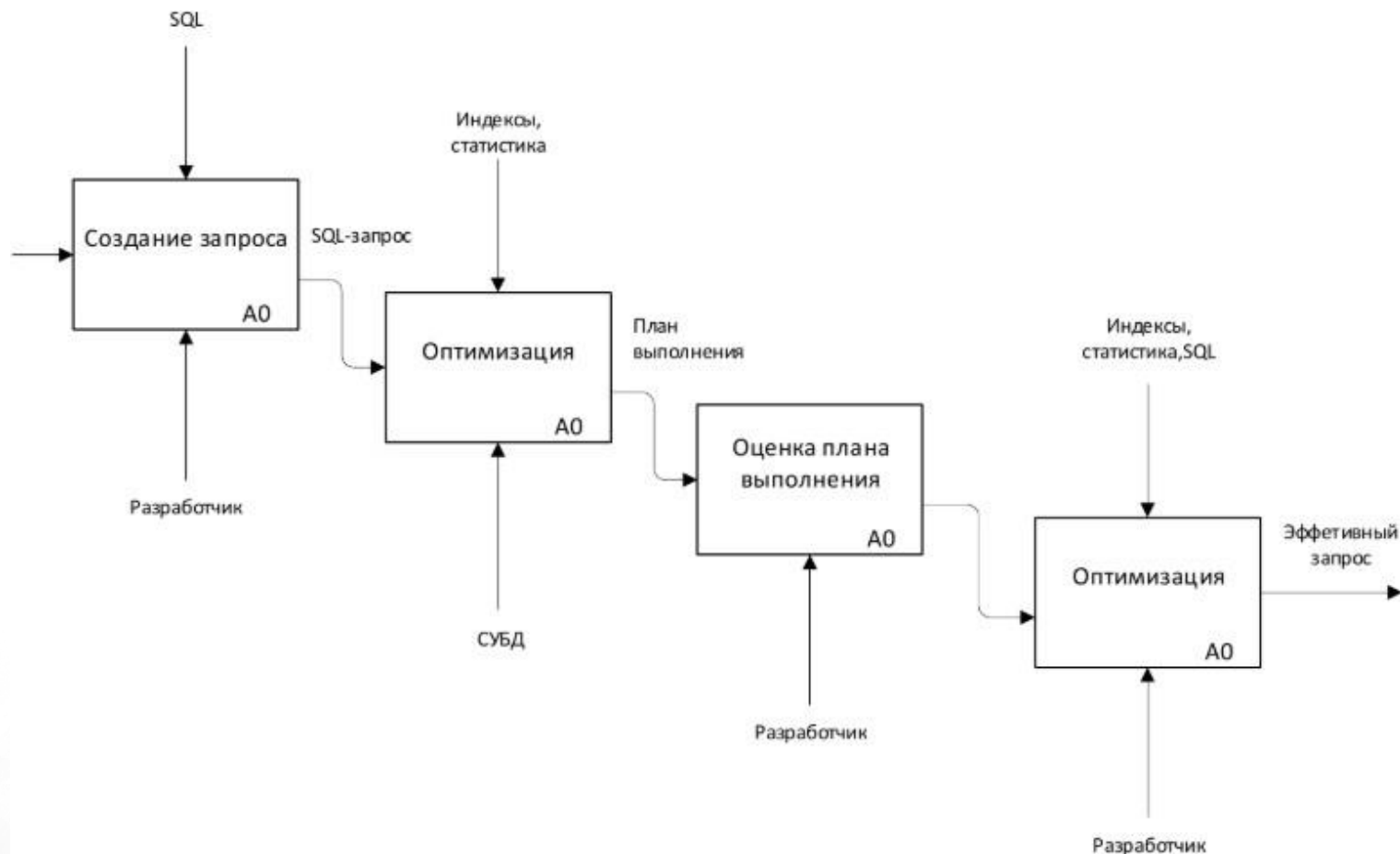
- Не рекомендуется создавать для:
 - ✓ столбцов с низкой селективностью
 - ✓ столбцов, имеющих много неопределенных значений (NULL-значения),
 - ✓ столбцов с часто изменяемыми значениями,
 - ✓ столбцов значительной длины (более чем 50 байт)
 - ✓ таблиц с небольшим количеством записей

Оптимизация SQL-запросов

Оптимизация запросов — набор действий, направленных на повышение эффективности запросов.

Выполняется:

- Средствами СУБД (поиск оптимального плана выполнения запроса).
- Разработчиком (изменение запроса, объектов БД, структуры БД).



Оптимизация одиночной инструкции SELECT

SQL-запрос является декларативным: в нем указывается, *какие данные* необходимы, но не *как их получить*.

Сервер базы данных должен проанализировать инструкцию для определения самого эффективного способа извлечения запрошенных данных.
Выполняющий это компонент называется оптимизатором запросов.

Работа оптимизатора:

На входе: Исходный запрос, схема БД (метаинформация о таблицах и индексах) и статистика базы данных

На выходе: План выполнения запроса — последовательность операций для максимально эффективного получения результата.

План выполнения запроса

План выполнения запроса — последовательность операций, необходимых для получения результата SQL-запроса.

План выполнения запроса определяет:

- Последовательность, в которой происходит обращение к исходным таблицам.
- Методы, используемые для извлечения и обработки данных.

Факторы, влияющие на план выполнения запроса

- использованные в запросе предикаты поиска,
- задействованные таблицы и условия соединения,
- элементы списка выборки,
- наличие полезных индексов, которые можно использовать для эффективного доступа к данным,
- актуальная статистика

Стоимость выполнения запроса

- Каждому возможному плану соответствует некоторая стоимость определенная в терминах объема использованных вычислительных ресурсов.
- Оптимизатор запросов должен проанализировать возможные планы и выбрать один план с самой низкой предполагаемой стоимостью.
- Оптимизатор запросов не анализирует все возможные комбинации, он старается найти достаточно хороший план за приемлемое время.

План выполнения запроса

План выполнения запроса PostgreSQL состоит из логических и физических операторов.

Логические операторы: определяют операции реляционной алгебры, используемые для выполнения инструкции

Физические операторы: Это конкретные операции, которые СУБД будет выполнять на диске и в памяти для получения результата. Например:

Сканирование таблицы (Seq Scan, Index Scan и т.д.).

Методы соединения таблиц(Hash Join, Nested Loop Join) и т.д.

Логическую операцию можно реализовать с помощью нескольких физических операторов.

Физический оператор может реализовывать несколько логических операций.

Оптимизатор запросов выбирает наиболее эффективный физический оператор для каждого логического.

Оператор EXPLAIN

Синтаксис EXPLAIN:

EXPLAIN [(параметр [, ...])] оператор

Здесь допускается параметр:

ANALYZE [boolean]

VERBOSE [boolean]

COSTS [boolean]

SETTINGS [boolean]

GENERIC_PLAN [boolean]

BUFFERS [boolean]

SERIALIZE [{ NONE | TEXT | BINARY }]

WAL [boolean]

TIMING [boolean]

SUMMARY [boolean]

MEMORY [boolean]

FORMAT { TEXT | XML | JSON | YAML }

В самой первой строке (основной строке самого верхнего узла) выводится общая стоимость выполнения для всего плана; именно это значение планировщик старается минимизировать:

Seq Scan on users	(cost=0.00..1834.00	rows=50000	width=44)		
↑	↑	↑	↑	↑	↑
операция	минимальная	полная	строки	средний размер	
	стоимость	стоимость		строки в байтах	

Пример плана выполнения запроса

```
EXPLAIN SELECT * FROM foo;
```

QUERY PLAN

```
-----  
Seq Scan on foo (cost=0.00..155.00 rows=10000 width=4)  
(1 row)
```

Стоимость родительского узла **включает в себя стоимость всех его дочерних узлов.**

Самый верхний узел в плане показывает общую стоимость всего запроса.

Главная задача планировщика — минимизировать общую стоимость запроса.

Seq Scan on foo — тип узла (последовательное сканирование таблицы foo).

(0.00): Стоимость запуска. Это подготовки данных, например, сортировка, перед тем, как узел начнет возвращать данные.

(155.00): Общая стоимость. Предполагаемая стоимость выполнения всего узла до конца.

rows = 10000 — ожидаемое количество строк, которое вернет этот узел.

width=4 — средний размер одной строки в байтах.

Пример плана выполнения запроса

```
EXPLAIN SELECT sum(i) FROM foo WHERE i < 10;
```

QUERY PLAN

```
-----  
Aggregate  (cost=23.93..23.93 rows=1 width=4)  
  -> Index Scan using fi on foo  (cost=0.00..23.92 rows=6 width=4)  
      Index Cond: (i < 10)  
(3 rows)
```

- На нижнем уровне используется индекс с именем fi для таблицы foo, по которому фильтруются строки, где значение столбца i меньше 10 (Index Condition: (i < 10))
- На верхнем уровне над результатами Index Scan применяется агрегатная функция sum(i).

Пример плана выполнения запроса

```
EXPLAIN SELECT *  
FROM tenk1 t1, tenk2 t2  
WHERE t1.unique1 < 10 AND t1.unique2 = t2.unique2;
```

QUERY PLAN

```
Nested Loop (cost=4.65..118.50 rows=10 width=488)  
-> Bitmap Heap Scan on tenk1 t1 (cost=4.36..39.38 rows=10 width=244)  
    Recheck Cond: (unique1 < 10)  
    -> Bitmap Index Scan on tenk1_unique1 (cost=0.00..4.36 rows=10 width=0)  
        Index Cond: (unique1 < 10)  
-> Index Scan using tenk2_unique2 on tenk2 t2 (cost=0.29..7.90 rows=1 width=244)  
    Index Cond: (unique2 = t1.unique2)
```

Пример плана выполнения запроса

Seq Scan - Операция сканирует всю таблицу в порядке, в котором она хранится на диске.

Index Scan - Сканирование индекса выполняет обход индекса, просматривает конечные узлы, чтобы найти все соответствующие записи, и извлекает соответствующие данные из таблицы.

Bitmap Index Scan - в то время как Index Scan извлекает один указатель кортежа за раз из индекса и немедленно переходит к этому кортежу в таблице. Bitmap Index Scan извлекает все указатели кортежей из индекса за один раз, сортирует их, используя структуру данных “битовая карта (bitmap)” в оперативной памяти, а затем просматривает кортежи таблиц в порядке физического расположения кортежей.

Merge Join - соединение слиянием объединяет два отсортированных списка.

Nested Loops - соединение вложенными циклами объединяет две таблицы, выбирая результат из одной таблицы и запрашивая другую таблицу для каждой строки из первой.

Hash Join - хеш-соединение загружает записи-кандидатов с одной стороны соединения в хеш-таблицу, которая затем проверяется для каждой строки с другой стороны соединения.

Sort - сортирует набор по столбцам, указанным в Sort Key.

Aggregate - появляется в плане, если в запросе есть агрегатная функция

GroupAggregate - группирует предварительно отсортированный набор в соответствии с предложением GROUP BY.

Пример плана выполнения запроса

HashAggregate - использует временную хэш-таблицу для группировки записей.

Unique - Удаляет дублирующиеся данные.

Filter – применяет фильтр к набору строк.

Limit - прерывает выполнение операций, когда было выбрано требуемое количество строк.

HashSetOp - операция используется операциями INTERSECT и EXCEPT (с опциональным модификатором ALL).

Materialize - Операция получает данные из нижележащей операции и размещает их в памяти (или частично в памяти), чтобы ими можно было быстрее воспользоваться

CTE Scan - схожа с Materialize. Операция выполняет часть запроса и сохраняет ее вывод, чтобы он мог быть использован другой частью (или частями) запроса.

SubPlan - операция означает подзапрос, в котором есть ссылки на основной запрос.

InitPlan - операция означает подзапрос, у которого нет ссылок на основной запрос.

Subquery Scan - Операция означает подзапрос, входящий в UNION.

Оптимизация запросов: правильное построение запроса

- Чем короче и проще запрос – тем лучше.
- Используйте правильно построенные предикаты поиска.
- Для фильтрации записей используйте WHERE, а не HAVING.
- Указывайте в разделе WHERE начальные столбцы ключа составного индекса.
- Минимизируйте число обращений к таблице.
- Соединяйте таблицы в правильном порядке.
- Уменьшите количество вложенных запросов.
- Делайте правильный выбор, выбирая между IN, EXISTS, JOIN:
 - Если в основной выборке много строк, а в подзапросе мало, то лучше использовать IN.
 - Если в подзапросе сложный запрос, а в основной выборке относительно мало строк, то лучше использовать EXISTS.
 - Если оба запроса сложные, лучше использовать JOIN.

Возврат столбцов, которые не требуются в дальнейших запросах

-- Плохо

```
SELECT * FROM products;
```

-- Хорошо

```
SELECT name FROM products;
```


Использование множественных подзапросов в SELECT

SELECT

```
c.*,  
(SELECT COUNT(*) FROM orders WHERE customer_id = c.customer_id) as order_count,  
(SELECT SUM(total_amount) FROM orders WHERE customer_id = c.customer_id) as total_spent,  
(SELECT MAX(order_date) FROM orders WHERE customer_id = c.customer_id) as last_order  
FROM customers c;
```

Seq Scan on customers c (cost=0.00..3388.80 rows=40 width=1654)

SubPlan 1

-> Aggregate (cost=28.14..28.15 rows=1 width=8)

-> Seq Scan on orders (cost=0.00..28.12 rows=7 width=0)

Filter: (customer_id = c.customer_id)

SubPlan 2

-> Aggregate (cost=28.14..28.16 rows=1 width=32)

-> Seq Scan on orders orders_1 (cost=0.00..28.12 rows=7 width=16)

Filter: (customer_id = c.customer_id)

SubPlan 3

-> Aggregate (cost=28.14..28.15 rows=1 width=4)

-> Seq Scan on orders orders_2 (cost=0.00..28.12 rows=7 width=4)

Filter: (customer_id = c.customer_id)

Использование множественных подзапросов в SELECT

-- Хорошо

EXPLAIN SELECT

c.customer_id,

c.first_name,

c.last_name,

COUNT(o.order_id) **as** order_count,

SUM(o.total_amount) **as** total_spent,

MAX(o.order_date) **as** last_order_date

FROM customers c

LEFT JOIN orders o **ON** o.customer_id = c.customer_id

GROUP BY c.customer_id, c.first_name, c.last_name

HAVING COUNT(o.order_id) > 0;

HashAggregate (cost=53.78..54.38 rows=13 width=1080)

Group Key: c.customer_id

Filter: (count(o.order_id) > 0)

-> Hash Right Join (cost=10.90..39.28 rows=1450 width=1060)

Hash Cond: (o.customer_id = c.customer_id)

-> Seq Scan on orders o (cost=0.00..24.50 rows=1450 width=28)

-> Hash (cost=10.40..10.40 rows=40 width=1036)

-> Seq Scan on customers c (cost=0.00..10.40 rows=40 width=1036)

Использование функций в соединениях и условиях, блокирующих работу индекса

--Плохо

```
EXPLAIN SELECT
  c.customer_id,
  c.first_name,
  c.last_name,
  o.order_date,
  o.total_amount
FROM orders o
JOIN customers c ON LOWER(c.first_name) = LOWER('иван')
WHERE EXTRACT(YEAR FROM o.order_date) = 2025;
```

Nested Loop (cost=0.00..11.68 rows=1 width=1056)

-> Seq Scan on orders o (cost=0.00..1.07 rows=1 width=20)

Filter: (EXTRACT(year FROM order_date) = '2025'::numeric)

-> Seq Scan on customers c (cost=0.00..10.60 rows=1 width=1...)

Filter: (lower((first_name)::text) = 'иван'::text)

Использование функций в соединениях и условиях, блокирующих работу индекса

EXPLAIN SELECT

```
c.customer_id,  
c.first_name,  
c.last_name,  
o.order_date,  
o.total_amount  
FROM customers c  
JOIN orders o ON o.customer_id = c.customer_id  
WHERE c.first_name = 'Иван'  
      AND o.order_date >= '2025-01-01'  
      AND o.order_date < '2026-01-01'  
ORDER BY o.order_date DESC;
```

Sort (cost=7.26..7.27 rows=1 width=1056)

Sort Key: o.order_date DESC

-> Nested Loop (cost=0.14..7.25 rows=1 width=1056)

-> Seq Scan on orders o (cost=0.00..1.07 rows=1 width=24)

Filter: ((order_date >= '2025-01-01'::date) AND (order_date < '2026-01-01'::date))

-> Index Scan using customers_pkey on customers c (cost=0.14..3.16 rows=1 width=24)

Index Cond: (customer_id = o.customer_id)

Filter: ((first_name)::text = 'Иван'::text)

Оптимизация запросов: использование индекса

Создать для

уникальных столбцов,
столбцов, участвующих в операции соединения,
столбцов с проверяемыми значениями,
столбцов, участвующих в агрегированных операциях

Запроса до создания индекса

```
EXPLAIN SELECT objectid, startdate FROM gar.apartments  
WHERE aparttype = (SELECT id FROM gar.apartment_types WHERE name='Квартира');
```

Seq Scan on apartments (cost=1.16..13899.34 rows=86216 width=12)

Filter: (aparttype = (InitPlan 1).col1)

InitPlan 1

-> Seq Scan on apartment_types (cost=0.00..1.16 rows=1 width=4)

Filter: ((name)::text = 'Квартира'::text)

после создания индекса

```
CREATE INDEX aparttype_idx ON gar.apartments(aparttype);  
EXPLAIN SELECT objectid, startdate FROM gar.apartments  
WHERE aparttype = (SELECT id FROM gar.apartment_types WHERE name='Квартира');
```

Index Scan using aparttype_idx on apartments (cost=1.58..6875.23 rows=86216 width=1...

Index Cond: (aparttype = (InitPlan 1).col1)

InitPlan 1

-> Seq Scan on apartment_types (cost=0.00..1.16 rows=1 width=4)

Filter: ((name)::text = 'Квартира'::text)

Выводы

- Производительность запросов влияет на эффективность работы БД.
- Для анализа производительности используется план выполнения запроса
- Оптимизация запроса процесс, направленный на увеличение производительности запроса
- В первую очередь необходимо оптимизировать часто выполняющиеся запросы

Спасибо за внимание!